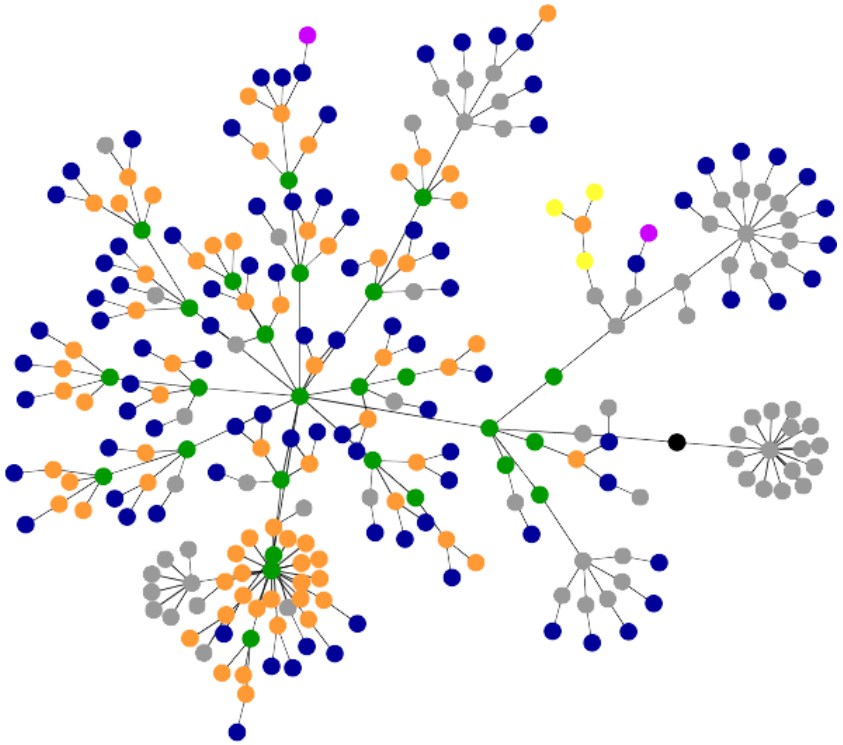
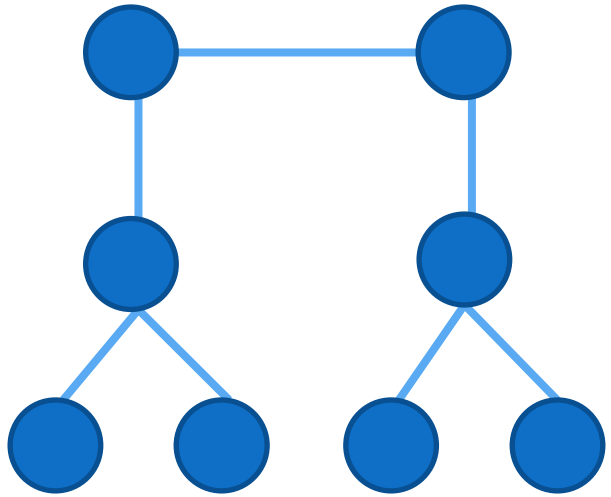


Árvores

Árvore

Grafo conexo e acíclico



Teorema de Caracterização de Árvores

1. *G* é uma árvore
2. Existe exatamente um caminho entre cada par de vértices de *G*
3. *G* é conexo e m=n-1 (m=número de arestas, n=número de vértices)
4. *G* é acíclico e m=n-1
5. *G* é acíclico e se quaisquer dois vértices não adjacentes de *G* forem conectados por uma aresta, então o grafo resultante conterá exatamente um ciclo.

Teorema de Caracterização de Árvores

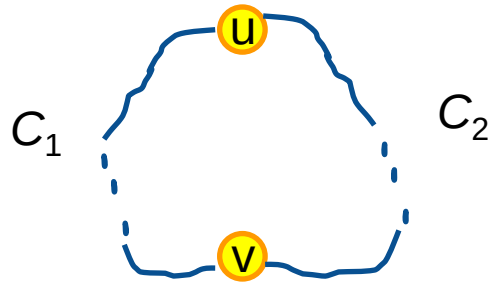
1. Se G é uma árvore
   então
2. Existe exatamente um caminho entre cada par de vértices de G

Como podemos provar isso?

Teorema de Caracterização de Árvores

1. Se G é uma árvore
   então
2. Existe exatamente um caminho entre cada par de vértices de G

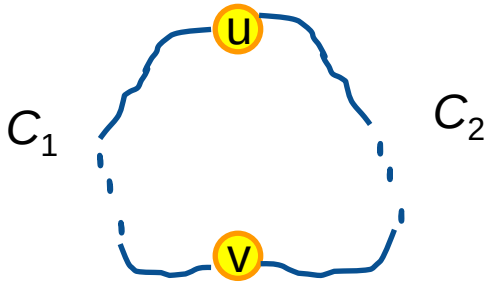
***Prova*: (por contradição)**
***Suponha que G* é uma árvore e existe mais de 1 caminho (caminhos distintos) entre algum par de vértices *u* e *v*. Suponha os caminhos C_1 e C_2.**



Teorema de Caracterização de Árvores

1. Se G é uma árvore
   então
2. Existe exatamente um caminho entre cada par de vértices de G

***Prova*: (por contradição)**
***Suponha que G* é uma árvore e existe mais de 1 caminho (caminhos distintos) entre algum par de vértices *u* e *v*. Suponha os caminhos C_1 e C_2.**



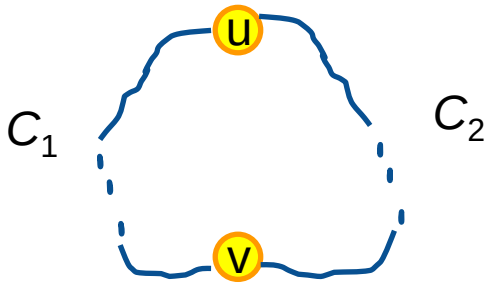
Teorema de Caracterização de Árvores

1. Se G é uma árvore
   então
2. Existe exatamente um caminho entre cada par de vértices de G

***Prova*: (por contradição)**
***Suponha que G* é uma árvore e existe mais de 1 caminho (caminhos distintos) entre algum par de vértices *u* e *v*. Suponha os caminhos C_1 e C_2.**

Então existe um ciclo passando por *u* e *v*.

Absurdo, porque árvore é um grafo acíclico.



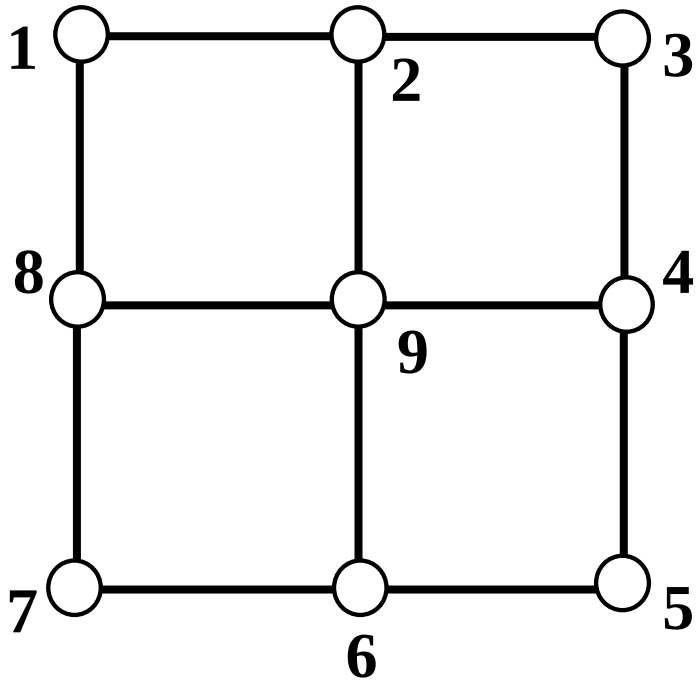
Árvore

Exercício

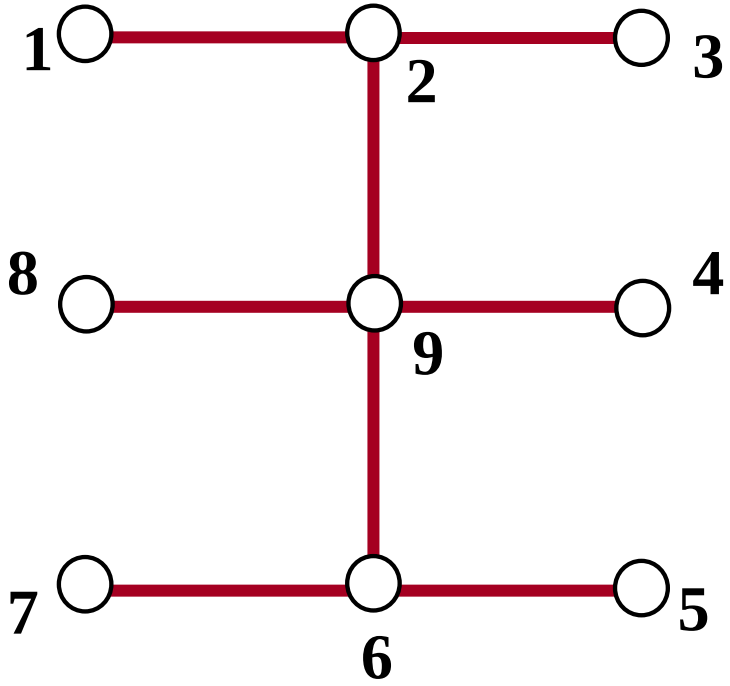
Provar o restante do Teorema de caracterização de árvores

Árvore Geradora

Uma ***Árvore Geradora*** de um grafo $G = (V,E)$ é um subgrafo gerador $T=(V,E_T)$ conexo e acíclico.



G



T

Árvores Geradoras

Considere G' um subgrafo gerador de G. Então as seguintes proposições são equivalentes:

1. ***G'* é uma árvore geradora de G**
2. **Existe exatamente um caminho entre cada par de vértices de *G'***
3. ***G'* é conexo e m=n-1**
4. ***G'* é acíclico e m=n-1**
5. ***G'* é acíclico e se quaisquer dois vértices não adjacentes de *G* forem conectados por uma aresta, então o grafo resultante conterá exatamente um ciclo.**

Árvores Geradoras

Teorema.

Um grafo G é conexo se e somente se possui árvore geradora.

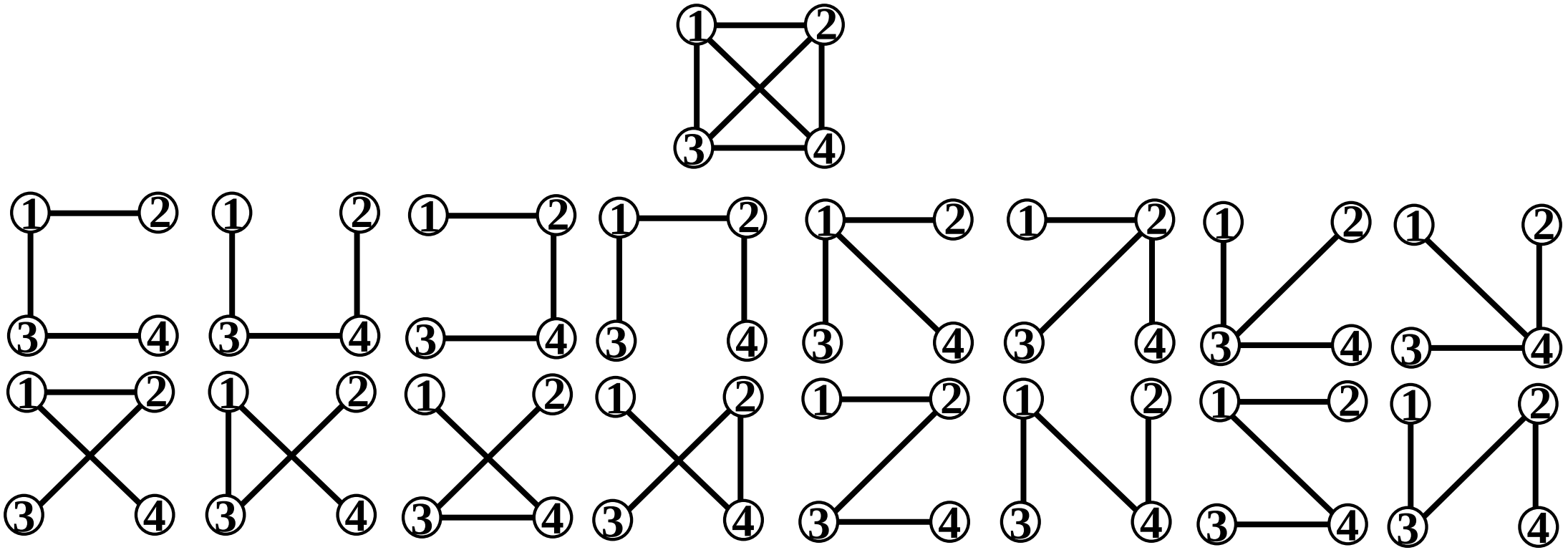
Provar o Teorema

Teorema de Cayley

Existem n^{n-2} árvores geradoras de grafos completos de ordem $n \geq 2$ rotulados em vértices.

Teorema de Cayley

***Existem n^{n-2}* árvores geradoras de grafos completos de ordem $n \geq 2$ rotulados em vértices.**



Teorema de Cayley

Existem n^{n-2} árvores geradoras em grafos completos de ordem $n \geq 2$ rotulados em vértices.

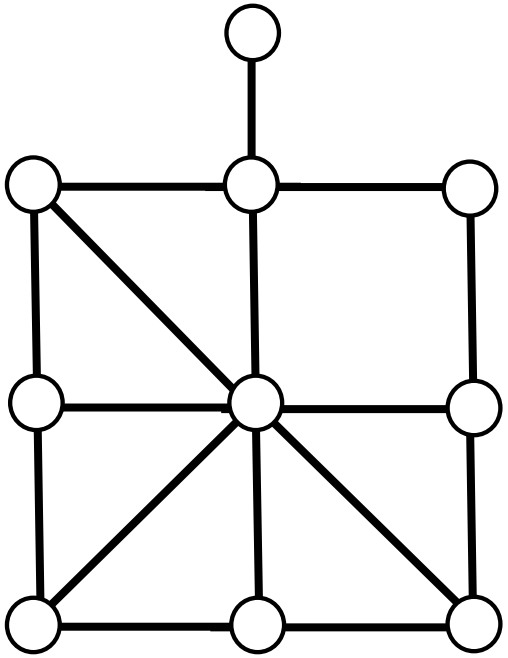
Provar o Teorema de Cayley

Dica: Utilize a codificação de Prüffer.

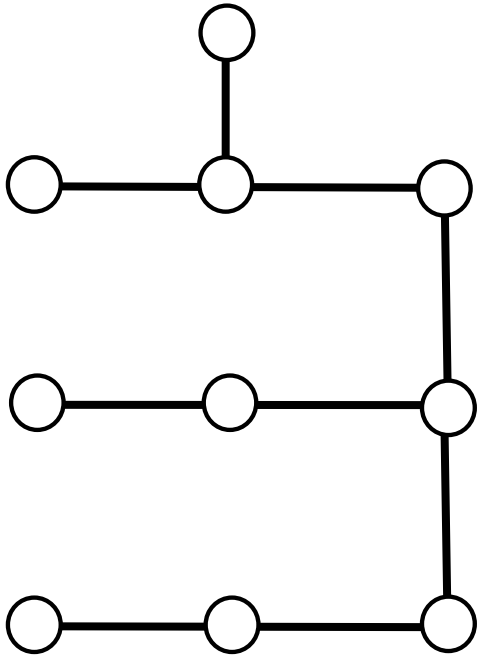
Obs. Este resultado não fornece apenas o número de árvores não isomorfas de K_n.

Co-Árvore Geradora

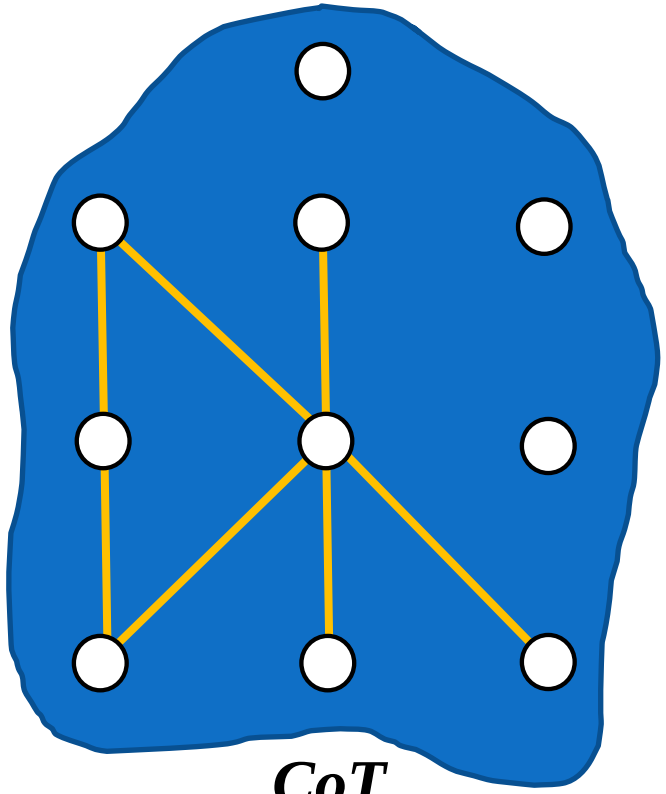
Dado $G=(V,E)$ e $T=(V,E_T)$, uma árvore geradora de G, o grafo complementar de T em relação a G é denominado *Co-Árvore Geradora*.



G



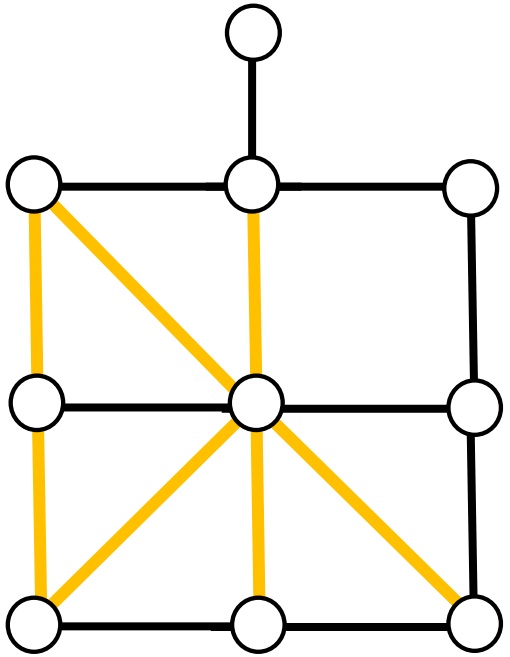
T



CoT

Co-Árvore Geradora

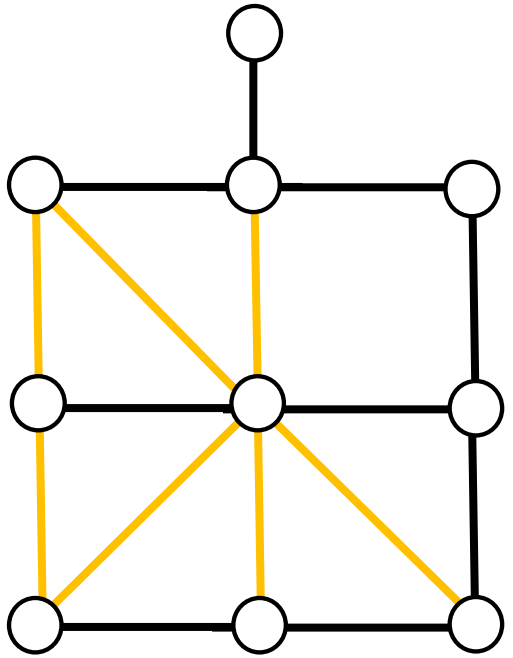
As arestas de uma co-árvore geradora de *G* são chamadas *elos*.



Co-Árvore Geradora

As arestas de uma co-árvore geradora de *G* são chamadas *elos*.

***número ciclomático de um grafo* = número de elos**

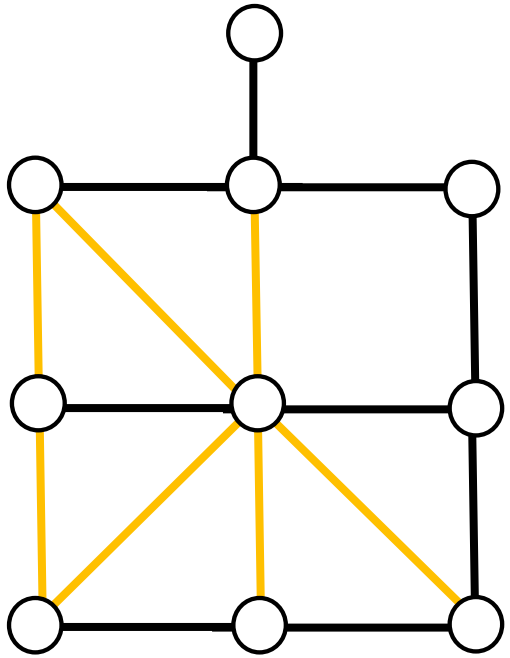


Co-Árvore Geradora

As arestas de uma co-árvore geradora de *G* são chamadas *elos*.

número ciclomático de um grafo = número de elos

número ciclomático(*G*) = número de arestas de *G* – número de arestas da árvore geradora



Co-Árvore Geradora

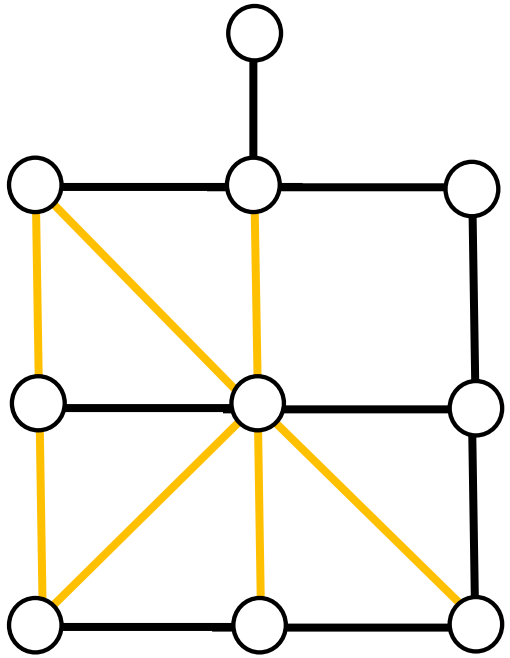
As arestas de uma co-árvore geradora de *G* são chamadas *elos*.

número ciclomático de um grafo = número de elos

número ciclomático(*G*) = número de arestas de *G* – número de arestas da árvore geradora

número ciclomático(*G*) = $m - (n - 1)$

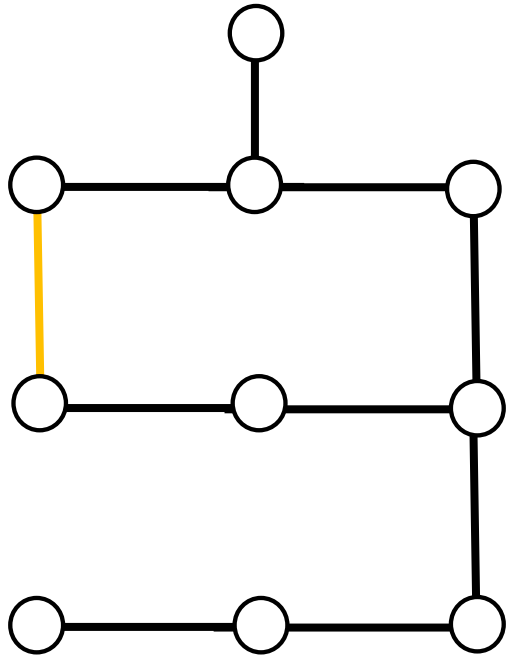
número ciclomático(*G*) = $m - n + 1$



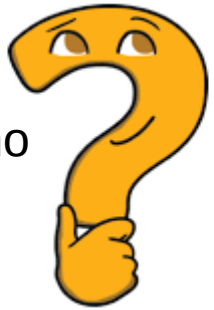
Ciclos Fundamentais

A adição de um *elo* a uma *árvore* produz um *único ciclo* no grafo resultante.

Tais ciclos são denominados *ciclos fundamentais*.



Quantos ciclos fundamentais existem no grafo?



k-árvore

Uma *k-árvore* é um grafo acíclico que contém *k* componentes.



k-árvore

Uma ***k-árvore*** é um grafo acíclico que contém *k* componentes.



Uma ***1-árvore*** é uma árvore.

Se uma *k*-árvore é um subgrafo gerador de um grafo *G*, então ela é dita ***k-árvore geradora***.



Rank e Nulidade

$G=(V,E)$, onde $|V| = n$ e $|E| = m$. G possui k componentes conexos.

$$rank(G) = n - k$$

$$nulidade(G) = m - n + k$$

Obs. rank(G) + nulidade(G) = m

Problema do Conector

Problema. Uma rede ferroviária conectando *n* cidades será construída. Dado o custo c_{ij} de construir um trecho de ferrovia que liga as cidades *i* e *j*, encontre uma configuração para esta rede tal que o custo total de construção seja minimizado.





custo c_{ij}

Problema do Conector

Problema. Uma rede ferroviária conectando *n* cidades será construída. Dado o custo c_{ij} de construir um trecho de ferrovia que liga as cidades *i* e *j*, encontre uma configuração para esta rede tal que o custo total de construção seja minimizado.

Modele este problema em grafos.

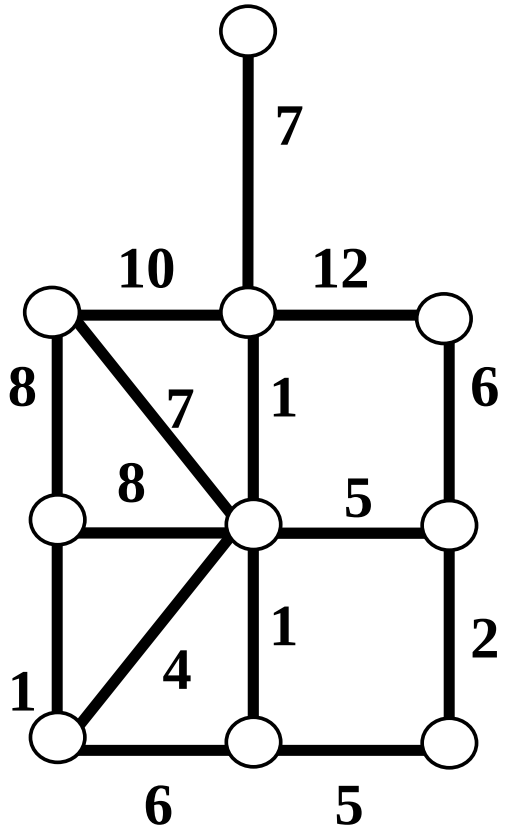
1. Considere a situação em que $c_{ij} = 1$ para todos os pares (i,j)
2. Considere a situação em que $c_{ij} \geq 1$ para todos os pares (i,j)

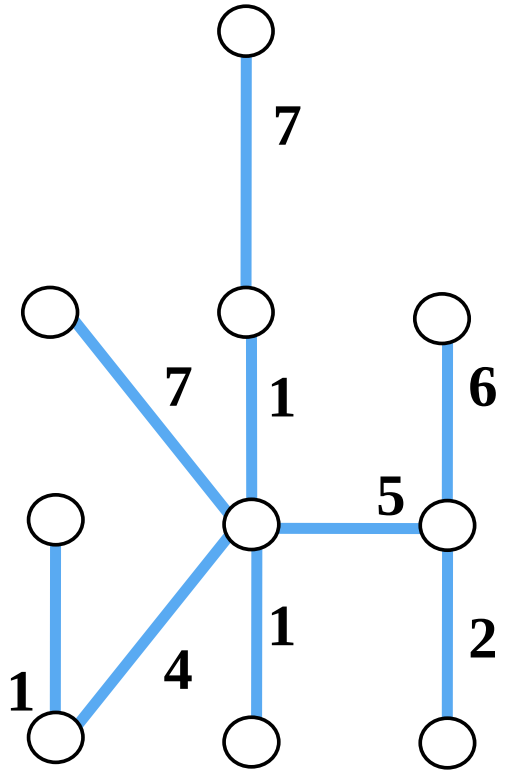
Apresente um algoritmo para solucioná-lo.



Árvore Geradora Mínima

Árvore geradora de menor custo, dentre todas as possíveis em um grafo ***G*** **ponderado em arestas.**





Custo = 34

Árvore Geradora Mínima

Um algoritmo para árvore geradora mínima



Como você faria?
Qual seria a complexidade?

Árvore Geradora Mínima

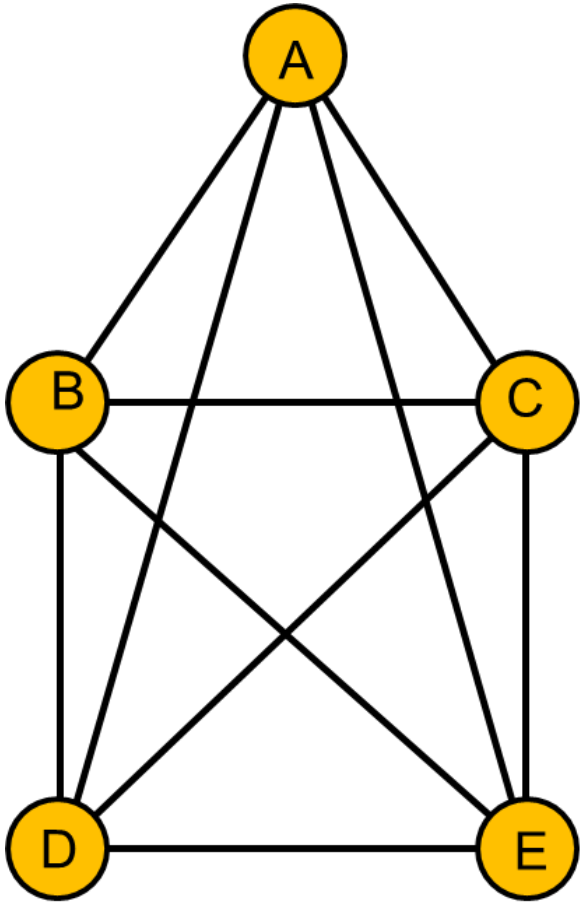
Como Kruskal (1956) pensou?

Ordena as *m* arestas do grafo.
Pega de uma em uma para incluir na árvore, desde de que não forme ciclo.
Pare quando tiver incluído *n*-1 arestas.

Kruskal, J. B. (1956). On the shortest spanning subtree of a graph and the traveling salesman problem. *Proceedings of the American Mathematical society*, *7*(1), 48-50.

Árvore Geradora Mínima

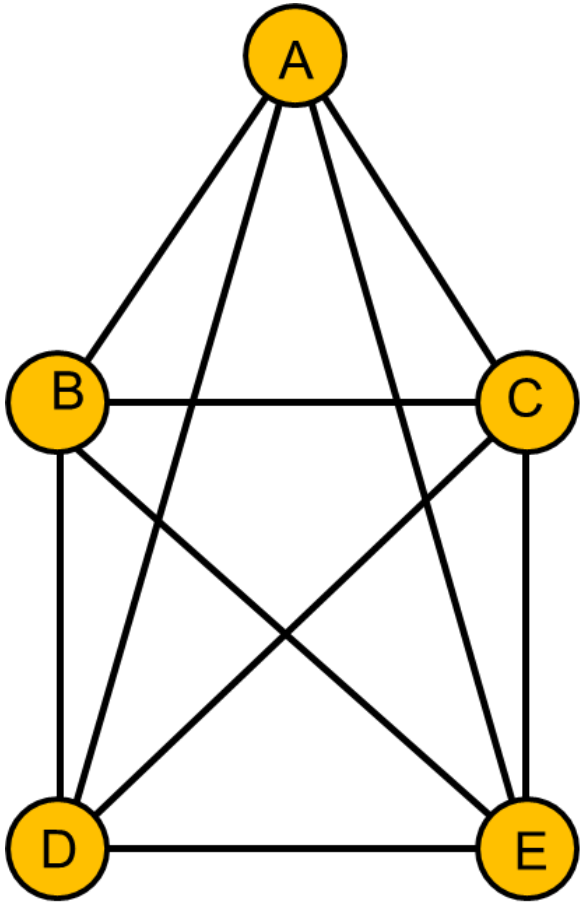
Método de Kruskal



Trecho	C_{IJ}
A,B	15
A,D	22
B,C	0
A,E	86
A,C	85
B,D	20
B,E	86
C,D	17
C,E	90
D,E	23

Árvore Geradora Mínima

Método de Kruskal



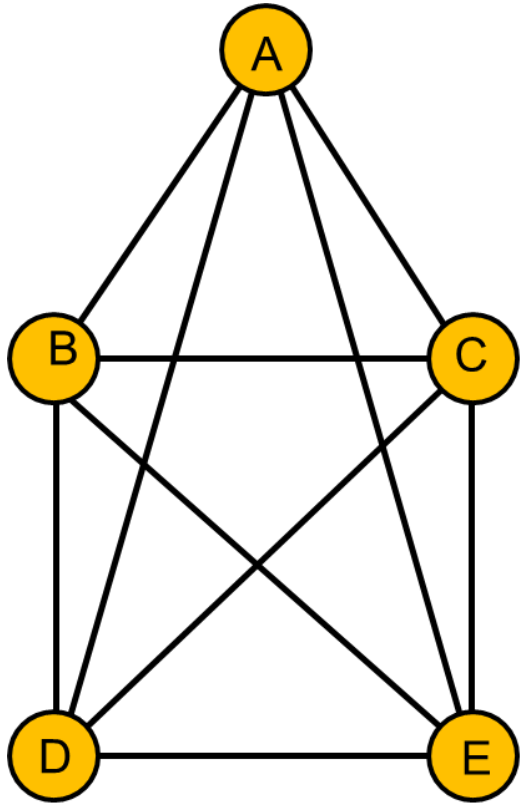
Trecho	C_{IJ}
A,B	15
A,D	22
B,C	0
A,E	86
A,C	85
B,D	20
B,E	86
C,D	17
C,E	90
D,E	23

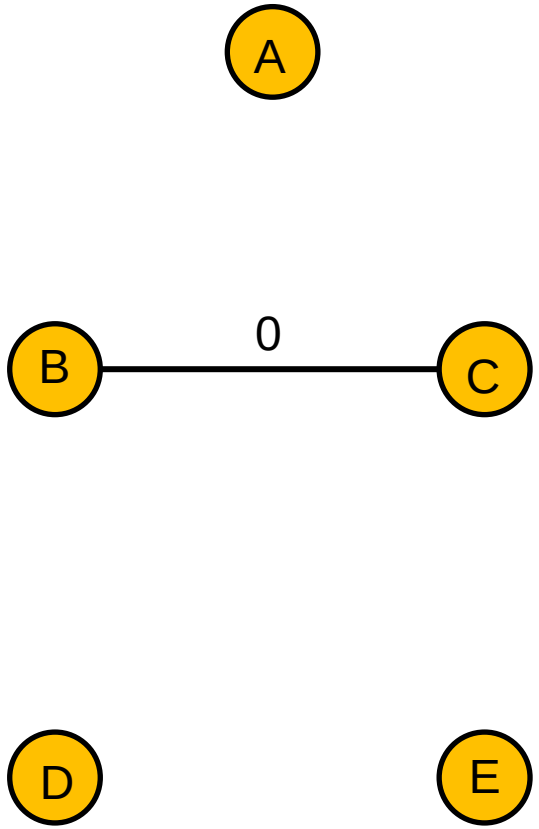
Ordena as arestas

Trecho	C_{IJ}
B,C	0
A,B	15
C,D	17
B,D	20
A,D	22
D,E	23
A,C	85
B,E	86
A,C	85
C,E	90

Árvore Geradora Mínima

Método de Kruskal

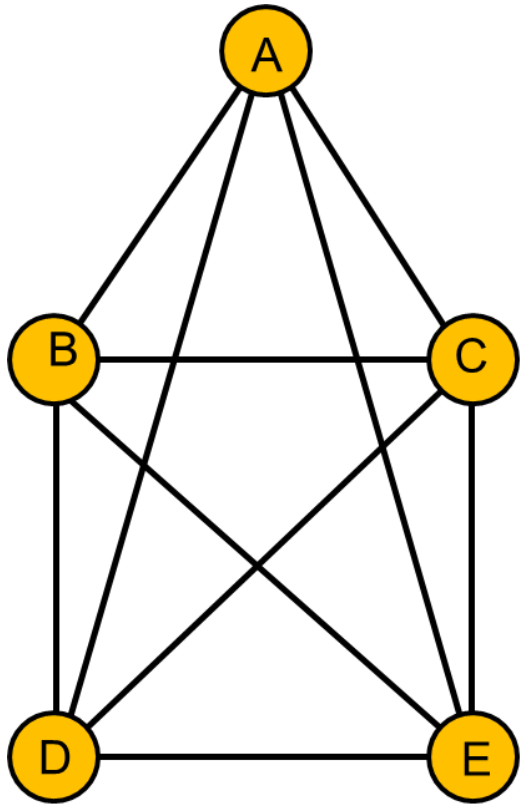


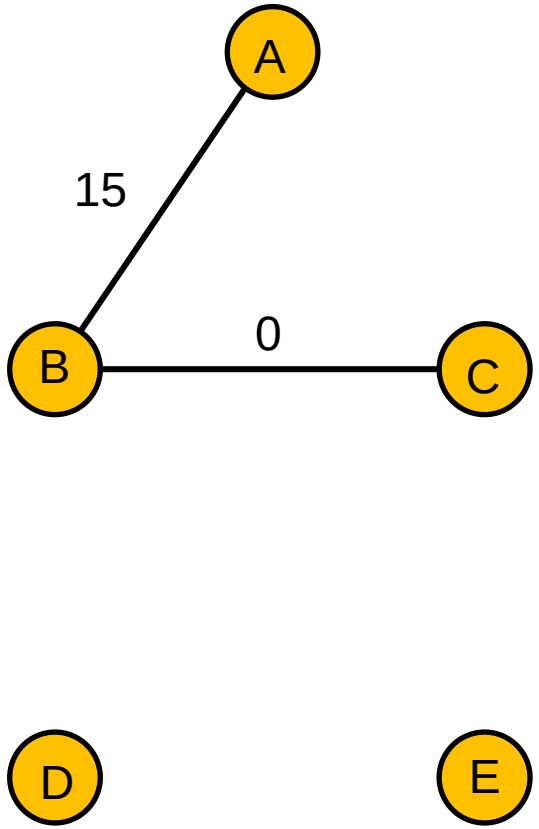


Trecho	C_{IJ}
~~B,C~~	~~0~~
A,B	15
C,D	17
B,D	20
A,D	22
D,E	23
A,C	85
B,E	86
A,C	85
C,E	90

Árvore Geradora Mínima

Método de Kruskal

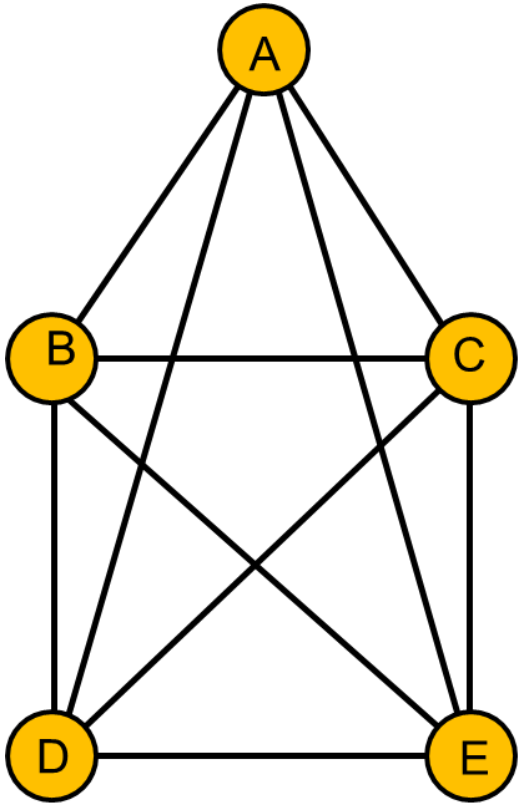


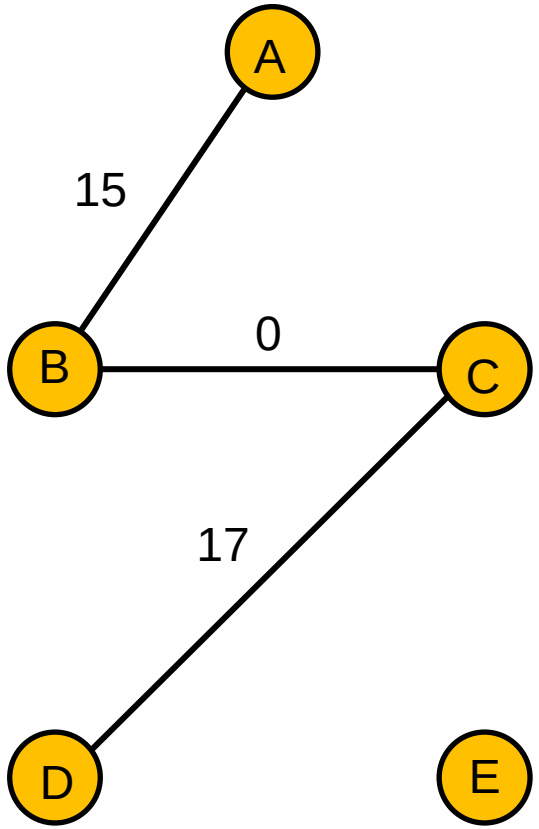


Trecho	C_{IJ}
~~B,C~~	~~0~~
~~A,B~~	~~15~~
C,D	17
B,D	20
A,D	22
D,E	23
A,C	85
B,E	86
A,C	85
C,E	90

Árvore Geradora Mínima

Método de Kruskal

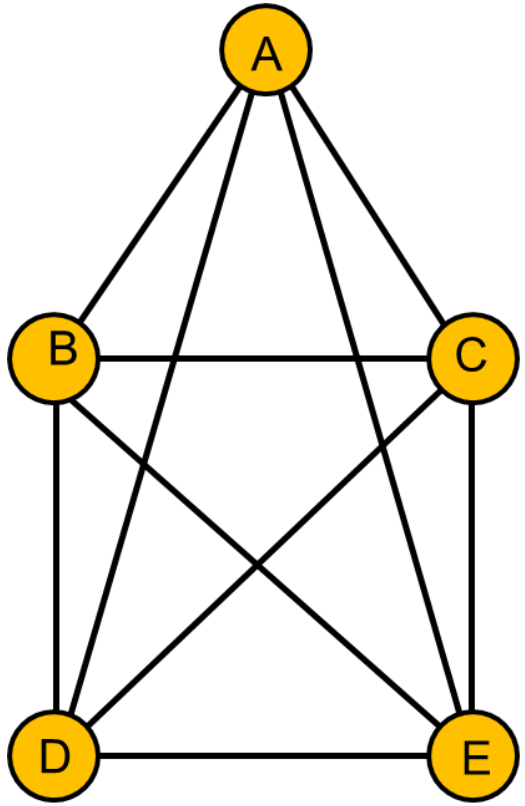


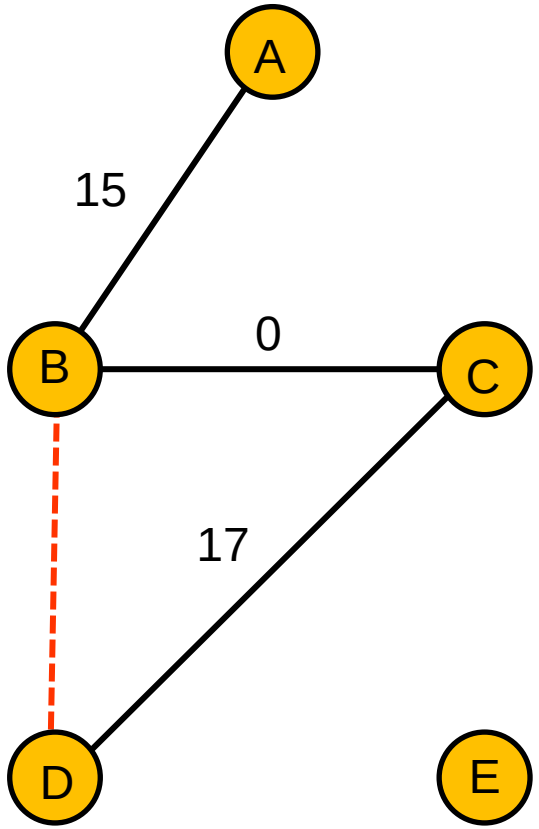


Trecho	C_{IJ}
~~B,C~~	~~0~~
~~A,B~~	~~15~~
~~C,D~~	~~17~~
B,D	20
A,D	22
D,E	23
A,C	85
B,E	86
A,C	85
C,E	90

Árvore Geradora Mínima

Método de Kruskal

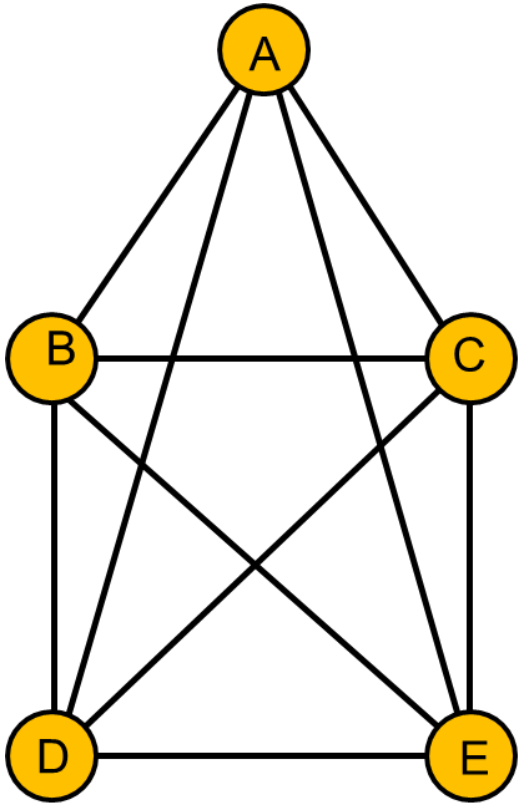


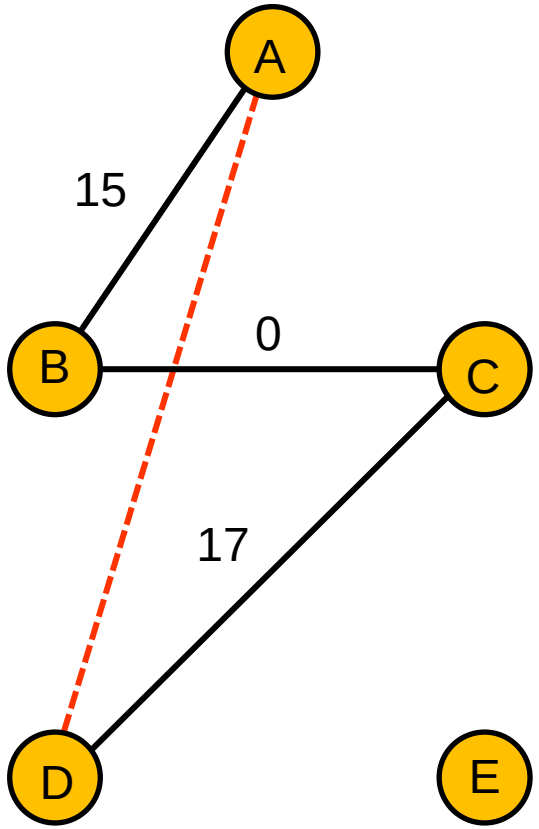


Trecho	C_{IJ}
~~B,C~~	~~0~~
~~A,B~~	~~15~~
~~C,D~~	~~17~~
~~B,D~~	~~20~~
A,D	22
D,E	23
A,C	85
B,E	86
A,C	85
C,E	90

Árvore Geradora Mínima

Método de Kruskal

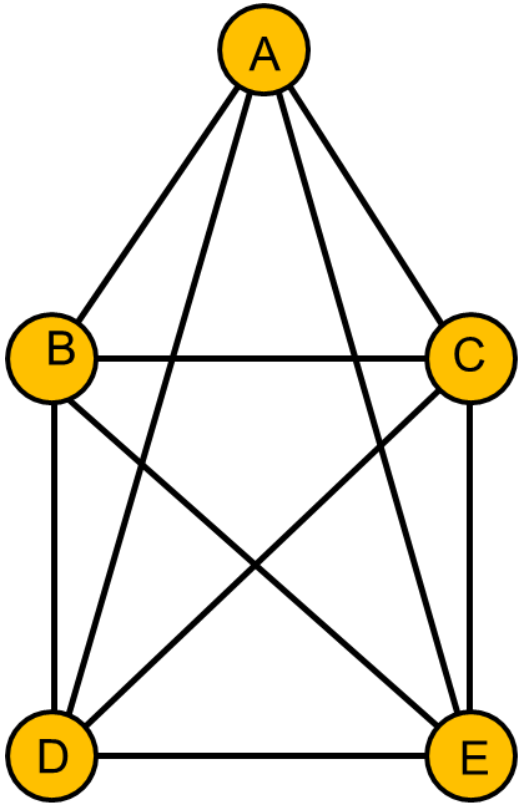


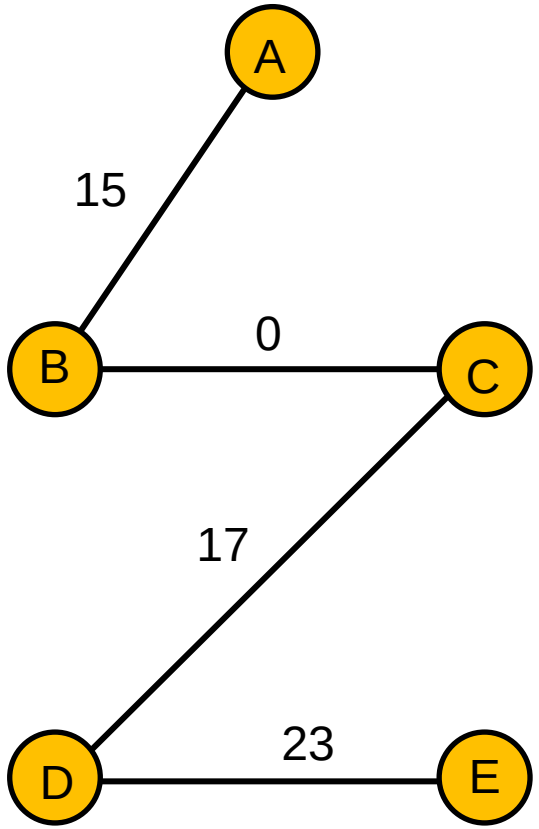


Trecho	C_{IJ}
~~B,C~~	~~0~~
~~A,B~~	~~15~~
~~C,D~~	~~17~~
~~B,D~~	~~20~~
~~A,D~~	~~22~~
D,E	23
A,C	85
B,E	86
A,C	85
C,E	90

Árvore Geradora Mínima

Método de Kruskal

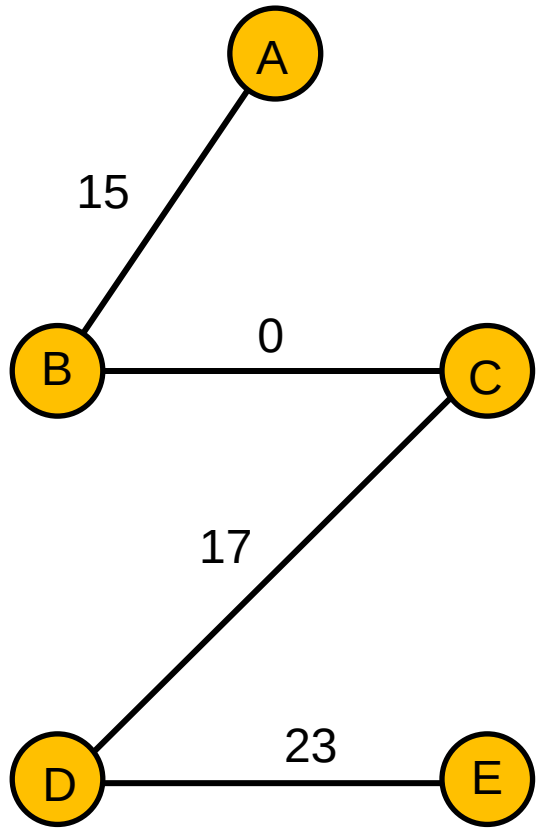




Trecho	C_{IJ}
~~B,C~~	~~0~~
~~A,B~~	~~15~~
~~C,D~~	~~17~~
~~B,D~~	~~20~~
~~A,D~~	~~22~~
~~D,E~~	~~23~~
A,C	85
B,E	86
A,C	85
C,E	90

Árvore Geradora Mínima

Método de Kruskal



Custo = 55

Algoritmo de Kruskal

O método de Kruskal sempre gera a árvore geradora mínima?
(Prova que o algoritmo é correto)

Qual a complexidade deste método?

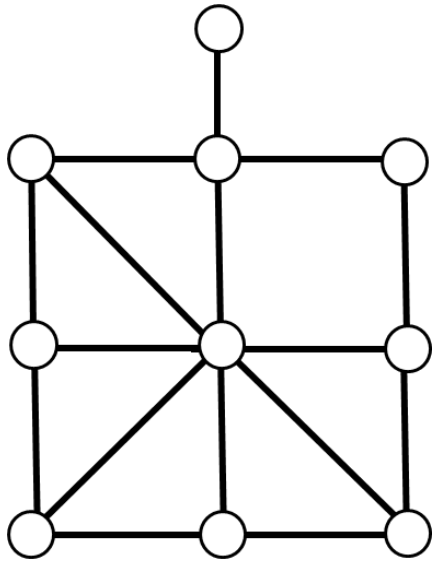
Árvore Geradora Mínima

Antes de responder estas perguntas, vejamos alguns resultados teóricos sobre as árvores geradoras mínimas.

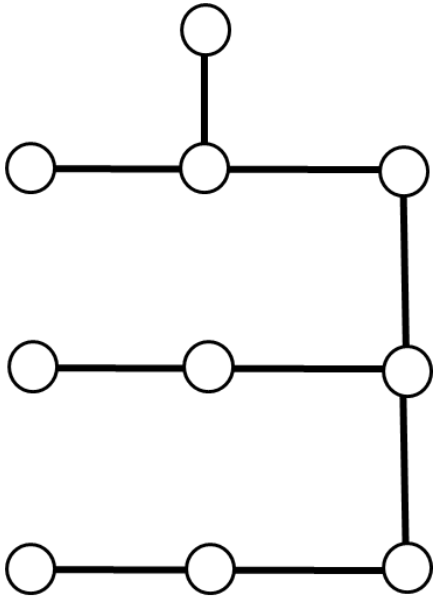
Veremos o algoritmo de Kruskal e outros adiante.

Árvore Geradora Mínima

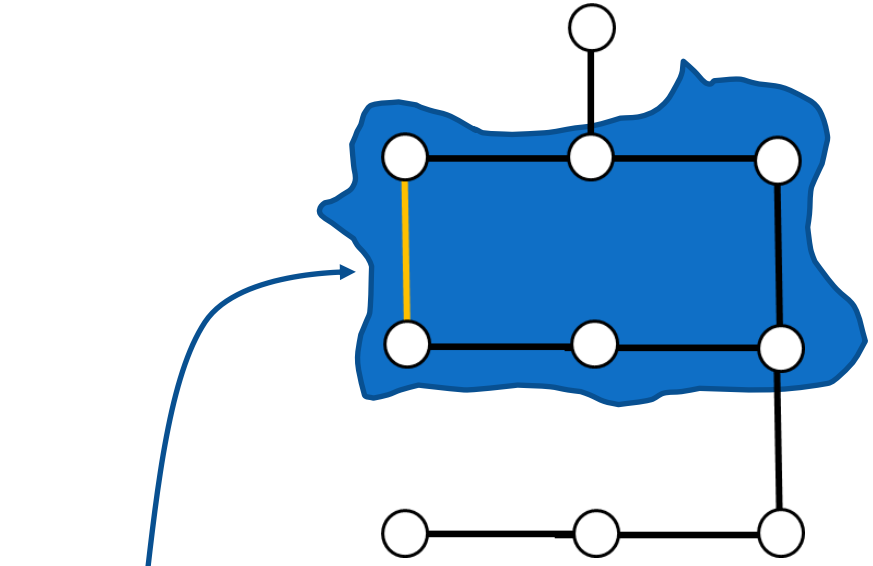
Dado um grafo G conexo e uma árvore geradora T de G, considere $C_T(e)$ o ciclo fundamental obtido pela adição do elo e à T e $w(e)$ o peso da aresta e.



G



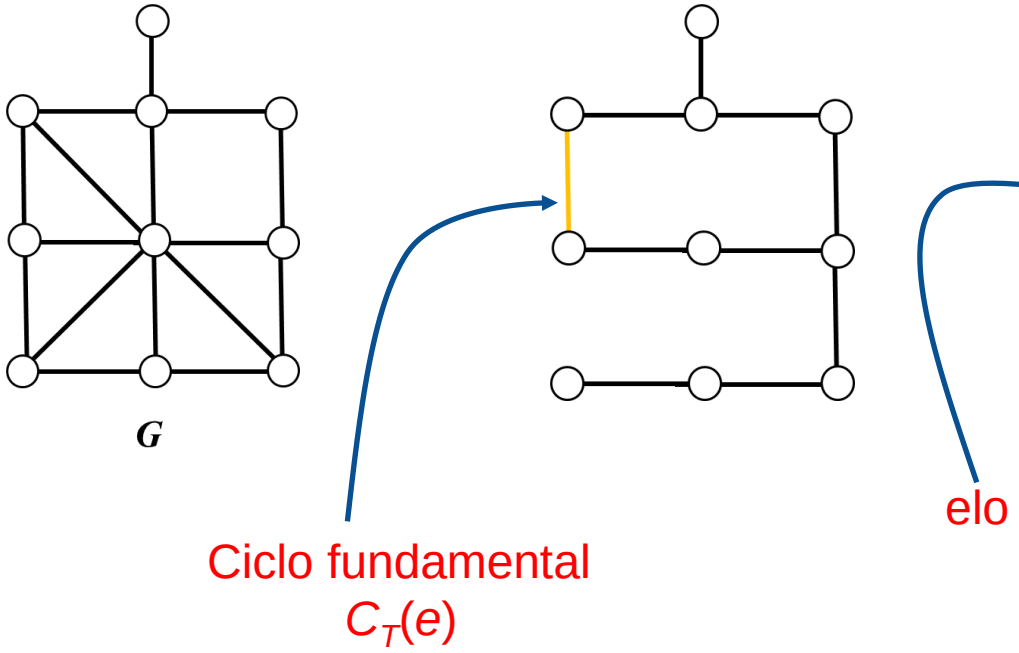
T



Ciclo fundamental
$C_T(e)$

Árvore Geradora Mínima

Dado um grafo G conexo e uma árvore geradora T de G, considere $C_T(e)$ o ciclo fundamental obtido pela adição do elo e à T e $w(e)$ o peso da aresta e.



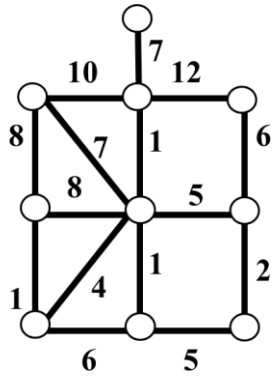
Teorema. Uma árvore geradora T de G é mínima se e somente se para cada aresta e em $G \setminus T$, tem-se $w(e) \geq w(f)$ para todas as arestas f em $C_T(e)$.

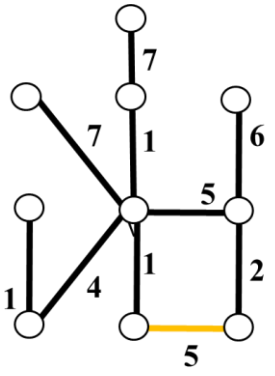
Árvore Geradora Mínima

Teorema. Uma árvore geradora T de G é mínima se e somente se para cada aresta e em $G\backslash T$, tem-se $w(e) \geq w(f)$ para todas as arestas f em $C_T(e)$.

Prova.

$\Rightarrow$Suponha que T é mínima e que existe aresta e tal que $w(e) < w(f)$ para alguma aresta f em $C_T(e)$. Removendo f de T e adicionando e, tem-se uma árvore T' com custo menor que T. Isso contradiz o fato de T ser mínima.





Árvore Geradora Mínima

Prova.

$\Leftarrow$ Suponha que $w(e) \geq w(f)$ para todas as arestas f em $C_T(e)$.

Suponha que o grafo G tem árvore geradora mínima T'. Devemos mostrar que custo(T') = custo(T), provando que T é árvore geradora mínima de G.

Prova por indução no número p de arestas em $T' \setminus T$.

Para $p = 0$, então $T' = T$, T é mínima.

Árvore Geradora Mínima

Prova.

$\Leftarrow$ Suponha que e' é uma aresta de $T' \setminus T$.

Se e' é removida de T', surgem duas componentes conexas T'_1 e T'_2.

$C_T(e') \setminus \{e'\}$ é o único caminho em T entre os vértices terminais de e'.

Se adicionarmos $C_T(e') \setminus \{e'\}$ a $T \setminus \{e'\}$, T'_1 e T'_2 são conectadas novamente.

Árvore Geradora Mínima

Prova.

$\Leftarrow$ Então $C_T(e')$ contém uma aresta e que liga um vértice de T'_1 a um vértice de T'_2.

e não pode ser uma aresta de T', caso contrário $T' \setminus \{e'\}$ seria conexo.

A minimalidade de T' implica que $w(e') \leq w(e)$.

Por outro lado, a condição do teorema diz que $w(e') \geq w(e)$. Logo, $w(e') = w(e)$.

Árvore Geradora Mínima

Prova.

$\Leftarrow$ Temos, então, que Custo(T'') = custo(T').

Portanto, T'' também é uma árvore geradora mínima de G.

T'' tem uma aresta a mais em comum com T do que T' e usando esta indução, temos

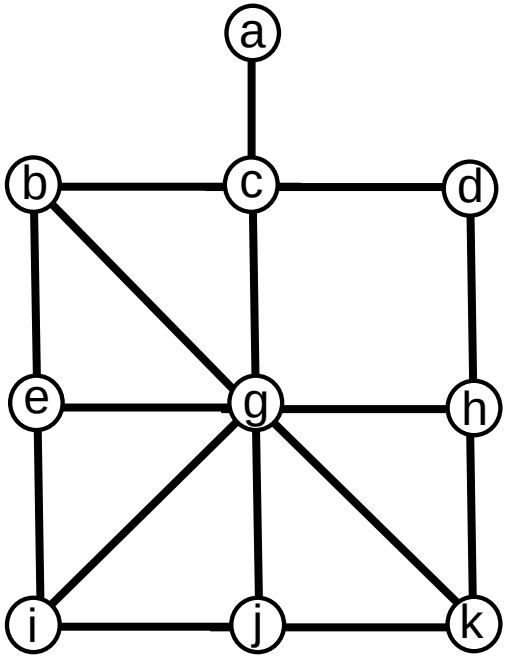
$$\text{Custo}(T) = \text{Custo}(T'') = \text{custo}(T').$$



Árvore Geradora Mínima

Considere ***G* = (*V*,*E*).**

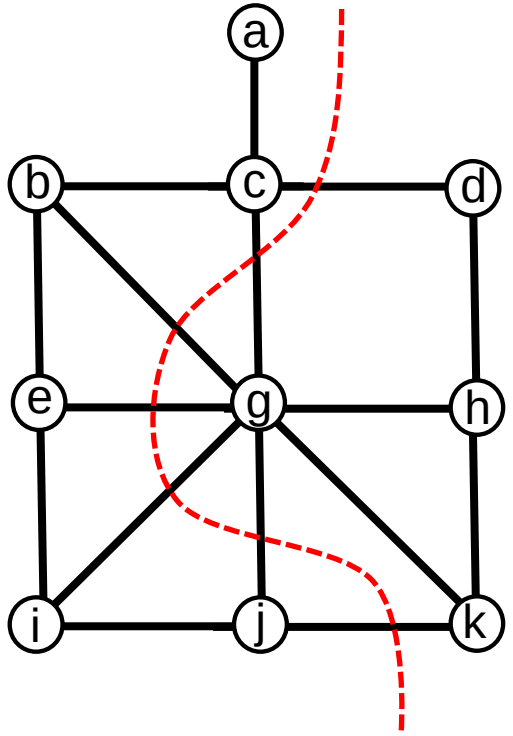
Um corte é uma partição $S = X \cup X'$ de ***V*** em 2 subconjuntos não vazios disjuntos.



Exemplo de corte:

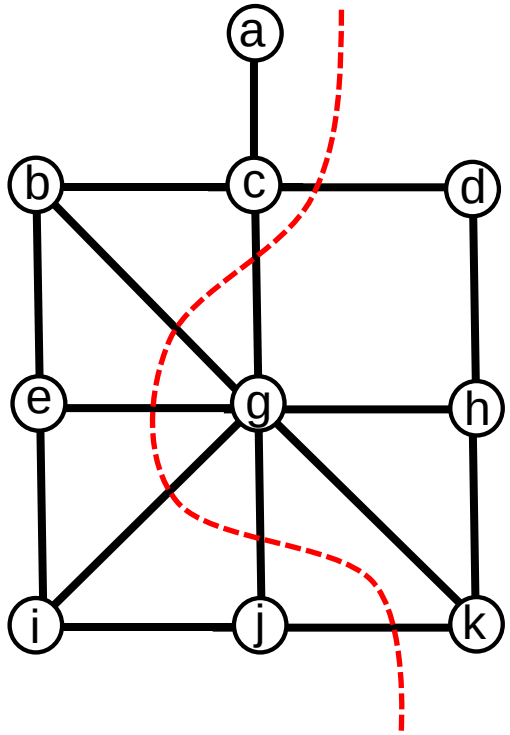
X = {a, b, c, e, i, j}

X' = {d, g, h, k}



Árvore Geradora Mínima

Considere $E(S)$ o conjunto de arestas que ligam um vértice de X a um vértice em X'. Um conjunto de tais arestas será chamado de co-ciclo.



X = {a, b, c, e, i, j}

X' = {d, g, h, k}

Co-ciclo

$E(S)$ = {(b,g), (c,d), (c,g), (e,g), (i,g), (g,j), (j,k)}

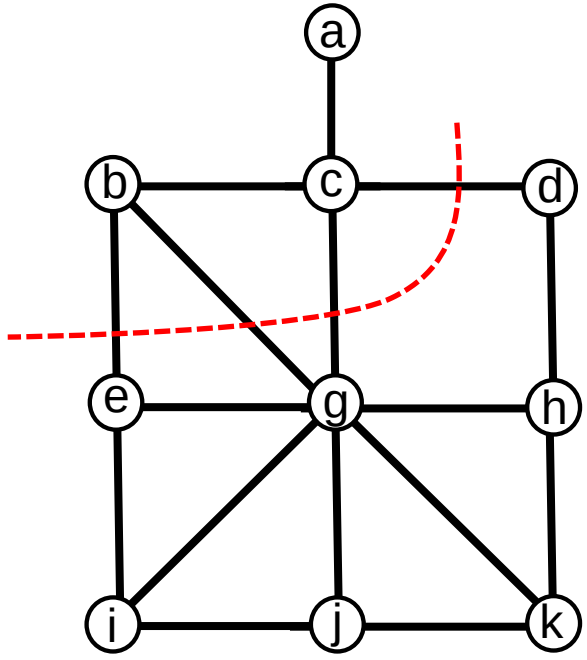
Árvore Geradora Mínima

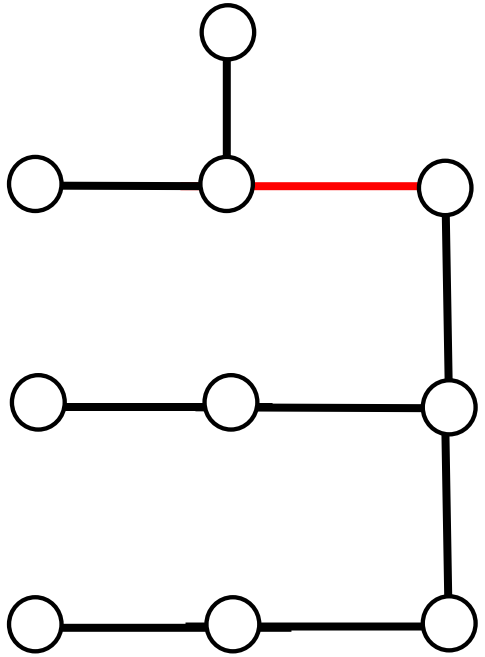
Lema. Seja G um grafo conexo e T uma árvore geradora de G. Para cada aresta e de T, existe exatamente um corte $S_T(e)$ de G tal que e é a única aresta que T tem em comum com o co-ciclo correspondente $E(S_T(e))$.

$e = (c,d)$

Co-ciclo

$E(S_T(e)) = \{(b,e), (b,g), (c,d), (c,g),\}$





Prova: Exercício

Árvore Geradora Mínima

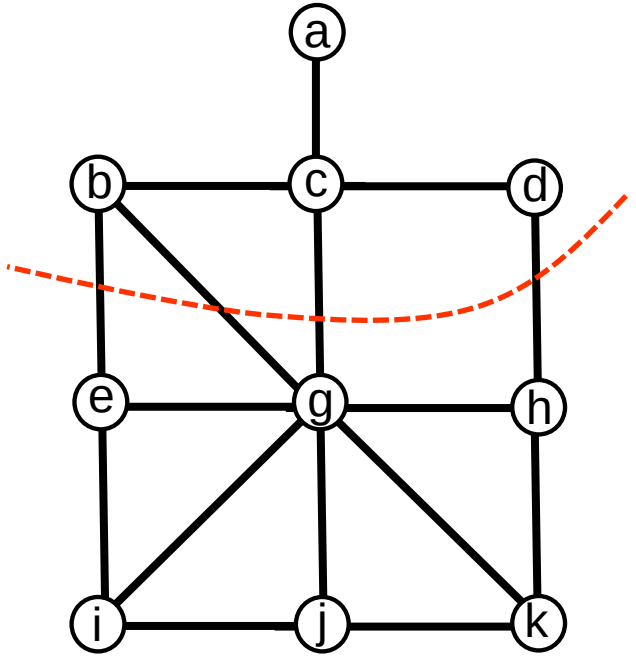
Teorema. Considere G um grafo conexo. Uma árvore geradora T de G é mínima se e somente se para cada aresta e de T, temos $w(e) \leq w(f)$ para cada aresta f em $E(S_T(e))$.

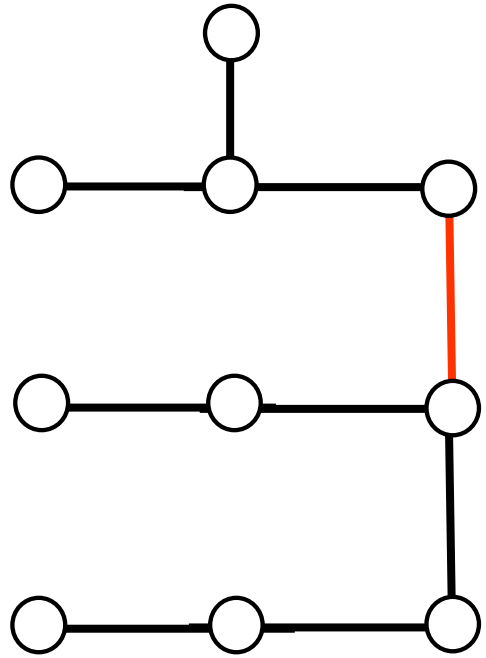
$e = (d,h)$

Co-ciclo

$E(S_T(e)) = \{(b,e), (b,g), (c,g), (d,h)\}$

Prova: Exercício





Árvore Geradora Mínima

Existe algoritmo polinomial para o problema da determinação da *árvore geradora mínima* de um grafo *G*.

Algoritmos clássicos:

- Kruskal
- Prim
- Borüvka



Algoritmo de Kruskal

Dados $G = (N, M)$, $D = [d_{ij}]$ a matriz de pesos das arestas de G.
Ordene as arestas de G em ordem não decrescente dos pesos d_{ij} no vetor H = (h_i), $i = 1,2,...,m$

$T \leftarrow \{h_1\}$

$i \leftarrow 2$

Enquanto $|T| < n-1$ **faça**

    Se $T \cup h_i$ é um grafo acíclico **então**

        $T \leftarrow T \cup \{h_i\}$

        $i \leftarrow i + 1$

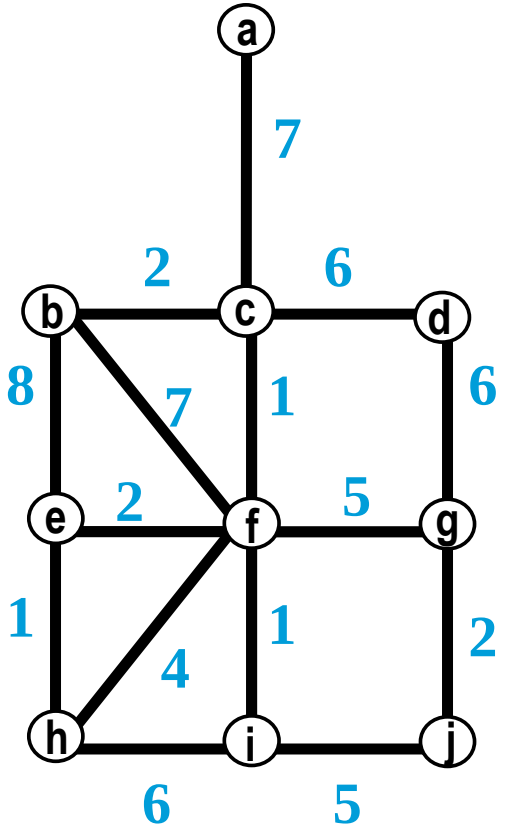
    Caso Contrário

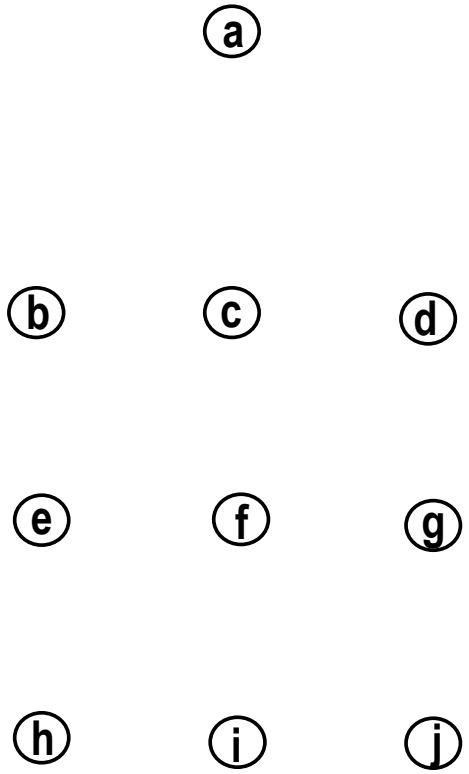
        $i \leftarrow i + 1$

Escrever T {arestas da árvore geradora mínima}

Algoritmo de Kruskal

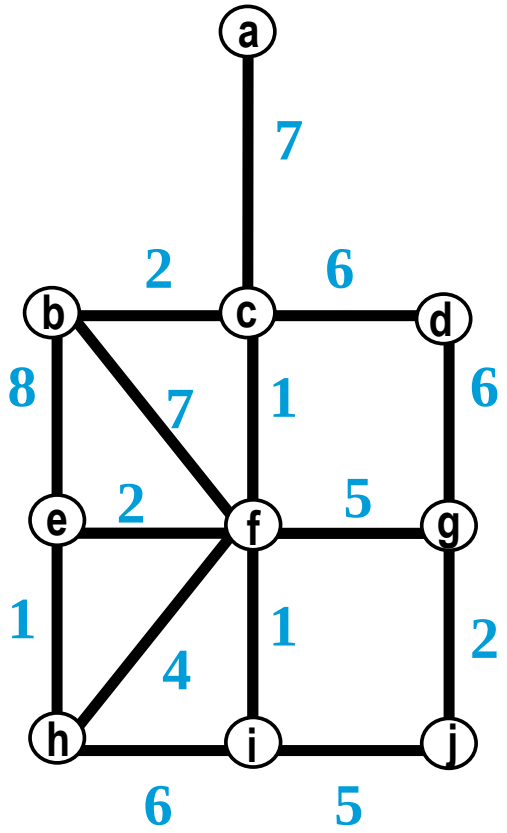
(c,f), (e,h), (f,i), (b,c), (e,f), (g,j), (f,h), (f,g), (i,j), (c,d), (d,g), (h,i), (a,c), (b,f), (b,e)

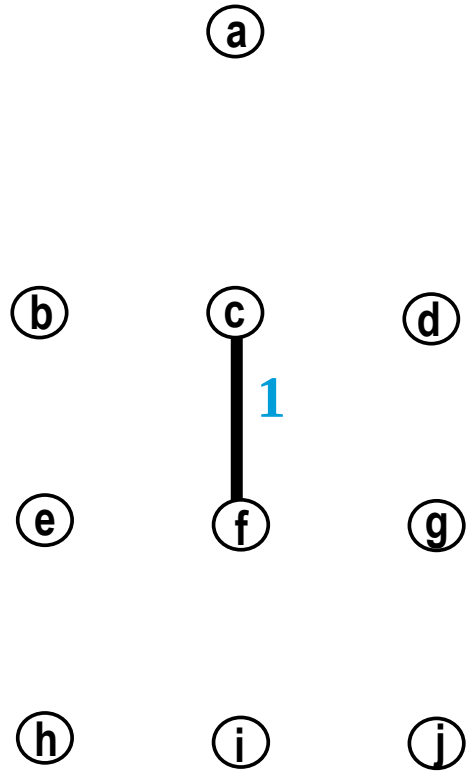




Algoritmo de Kruskal

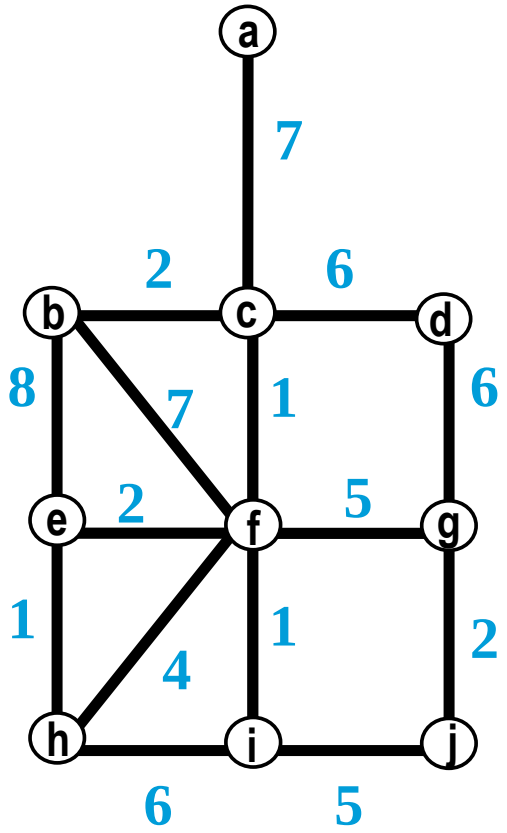
(c,f), (e,h), (f,i), (b,c), (e,f), (g,j), (f,h), (f,g), (i,j), (c,d), (d,g), (h,i), (a,c), (b,f), (b,e)

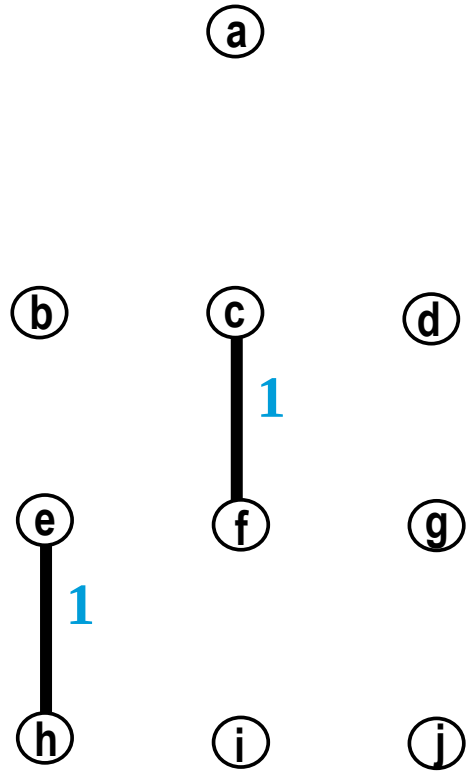




Algoritmo de Kruskal

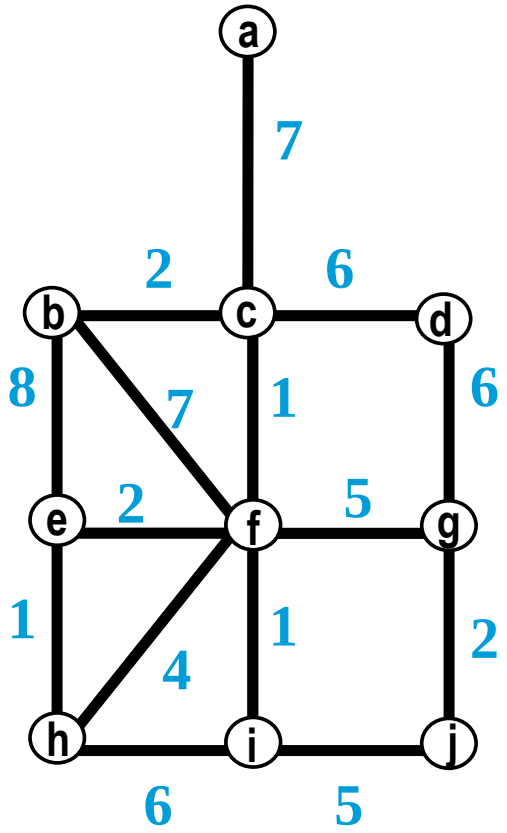
(c,f), (e,h), (f,i), (b,c), (e,f), (g,j), (f,h), (f,g), (i,j), (c,d), (d,g), (h,i), (a,c), (b,f), (b,e)

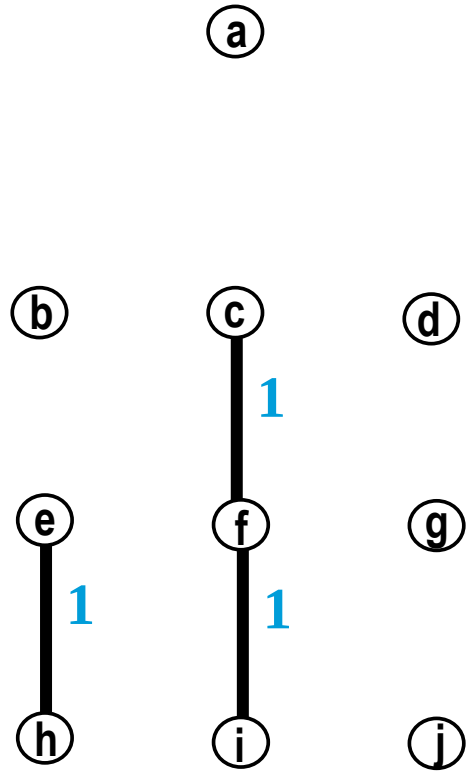




Algoritmo de Kruskal

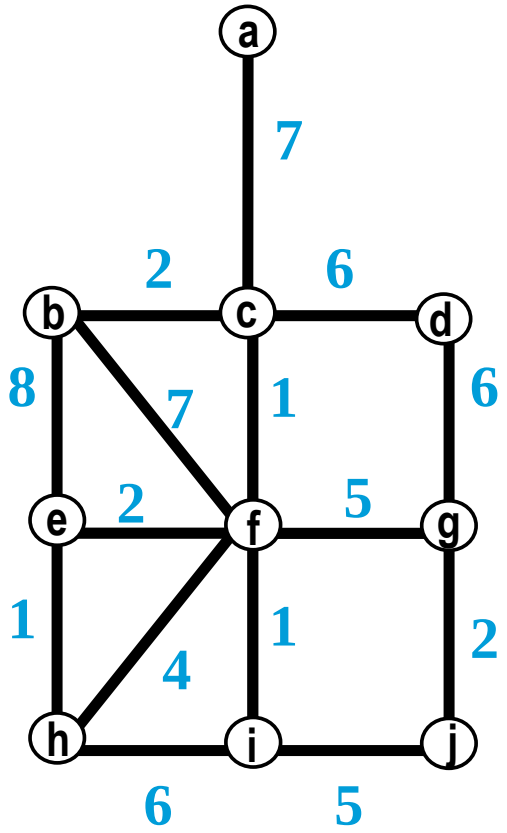
~~(c,f)~~, ~~(e,h)~~, ~~(f,i)~~, (b,c), (e,f), (g,j), (f,h), (f,g), (i,j), (c,d), (d,g), (h,i), (a,c), (b,f), (b,e)

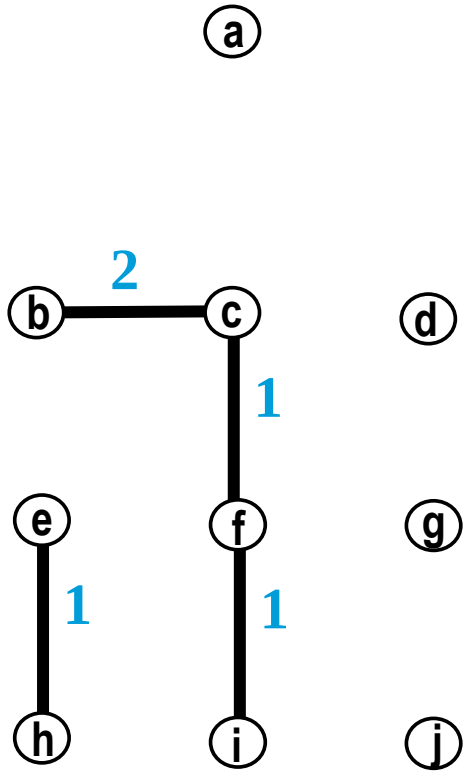




Algoritmo de Kruskal

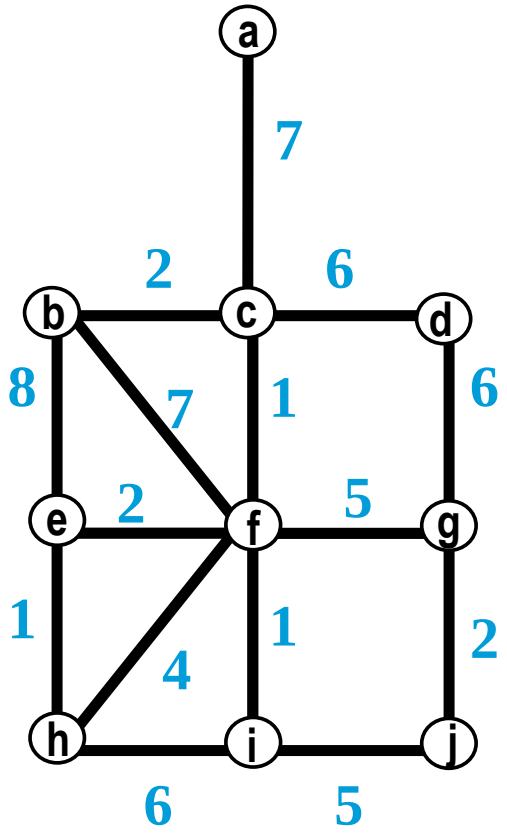
~~(c,f)~~, ~~(e,h)~~, ~~(f,i)~~, ~~(b,c)~~, (e,f), (g,j), (f,h), (f,g), (i,j), (c,d), (d,g), (h,i), (a,c), (b,f), (b,e)

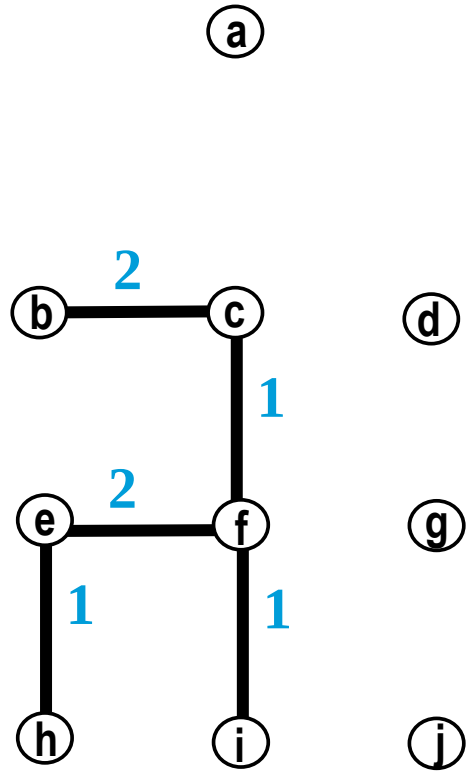




Algoritmo de Kruskal

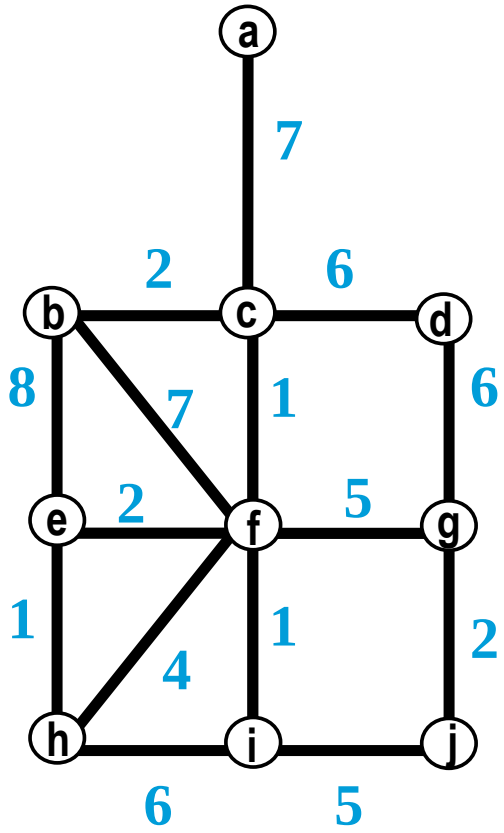
~~(c,f)~~, ~~(e,h)~~, ~~(f,i)~~, (b,~~c~~), (~~e~~,f), (g,j), (f,h), (f,g), (i,j), (c,d), (d,g), (h,i), (a,c), (b,f), (b,e)

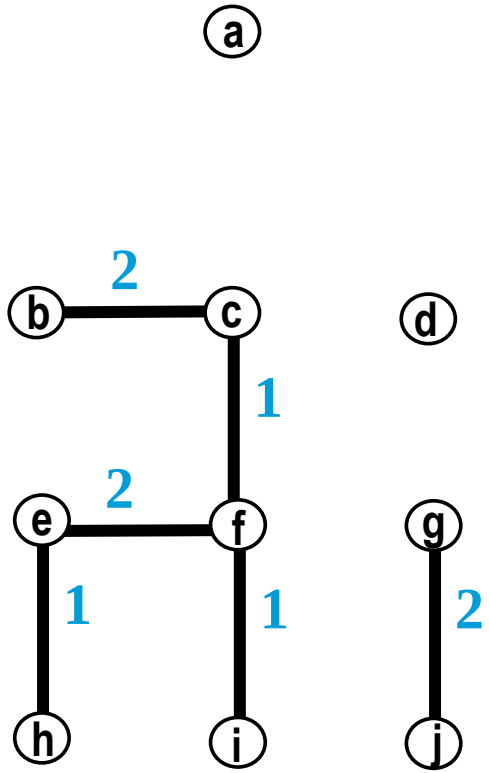




Algoritmo de Kruskal

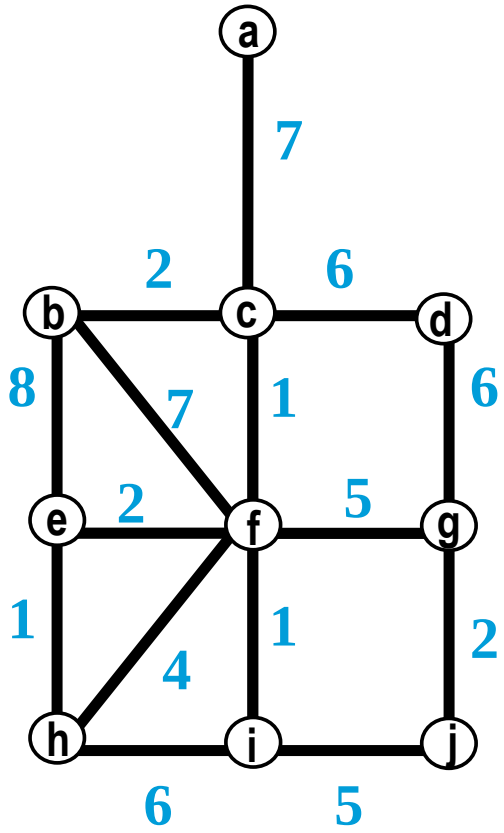
~~(c,f)~~, ~~(e,h)~~, ~~(f,i)~~, ~~(b,c)~~, ~~(e,f)~~, ~~(g,j)~~, (f,h), (f,g), (i,j), (c,d), (d,g), (h,i), (a,c), (b,f), (b,e)

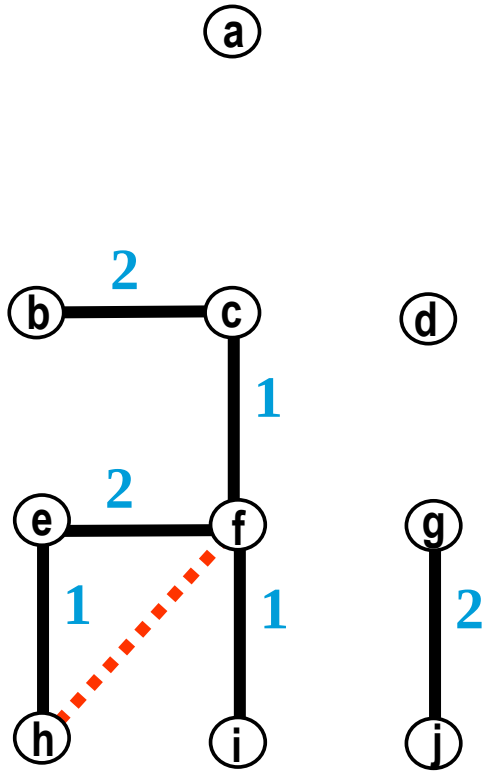




Algoritmo de Kruskal

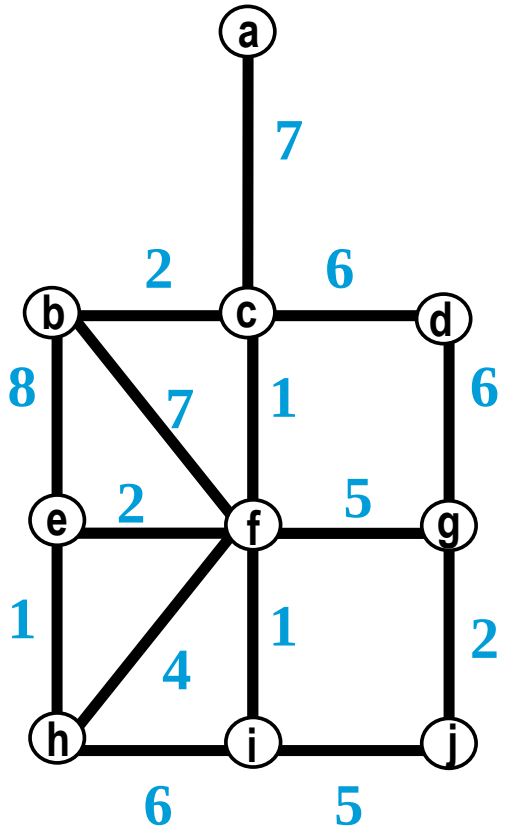
~~(c,f)~~, ~~(e,h)~~, ~~(f,i)~~, ~~(b,c)~~, ~~(e,f)~~, ~~(g,j)~~, ~~(f,h)~~, (f,g), (i,j), (c,d), (d,g), (h,i), (a,c), (b,f), (b,e)

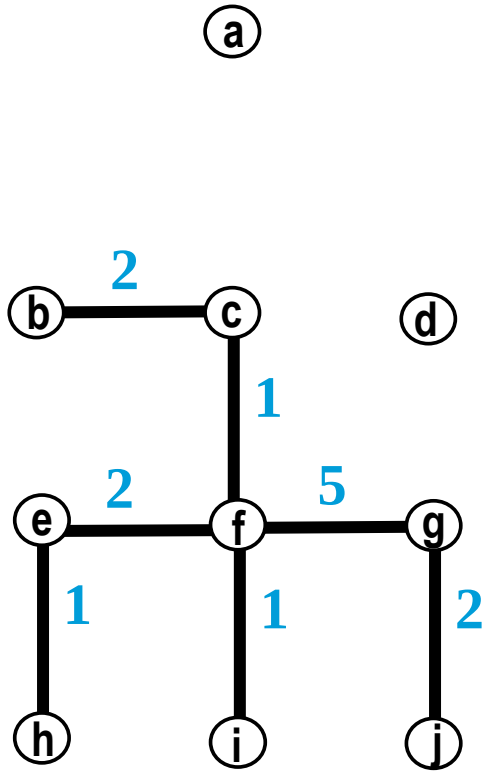




Algoritmo de Kruskal

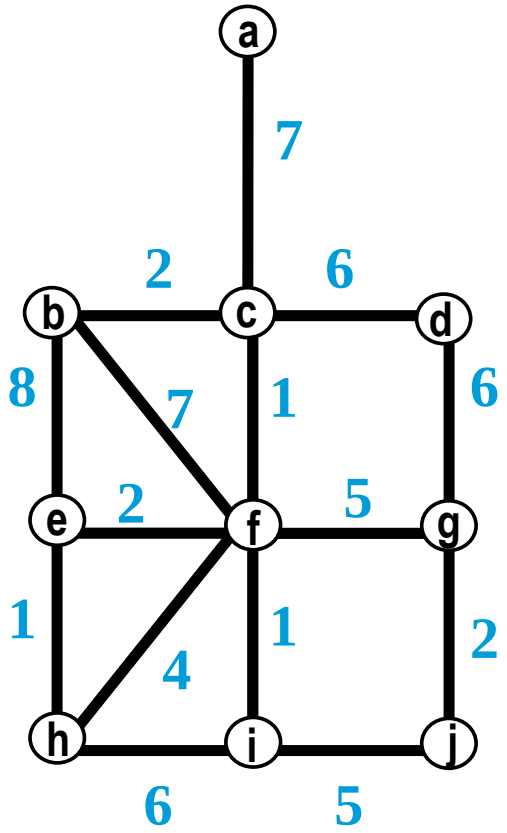
~~(c,f)~~, ~~(e,h)~~, ~~(f,i)~~, ~~(b,c)~~, ~~(e,f)~~, ~~(g,j)~~, ~~(f,h)~~, ~~(f,g)~~, (i,j), (c,d), (d,g), (h,i), (a,c), (b,f), (b,e)

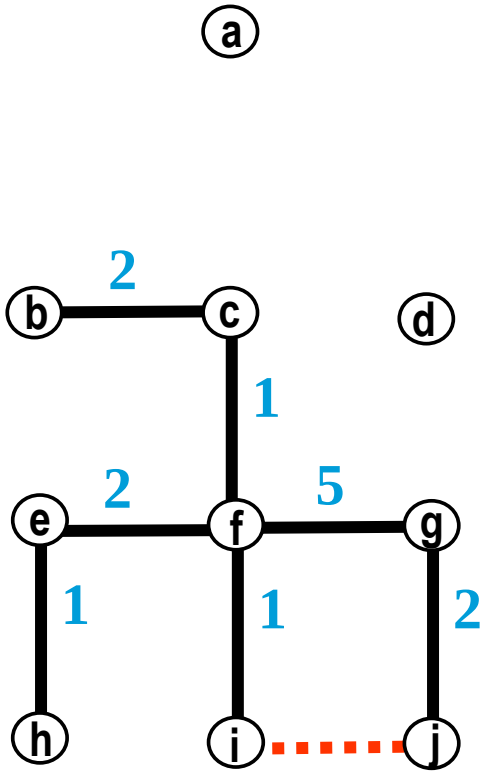




Algoritmo de Kruskal

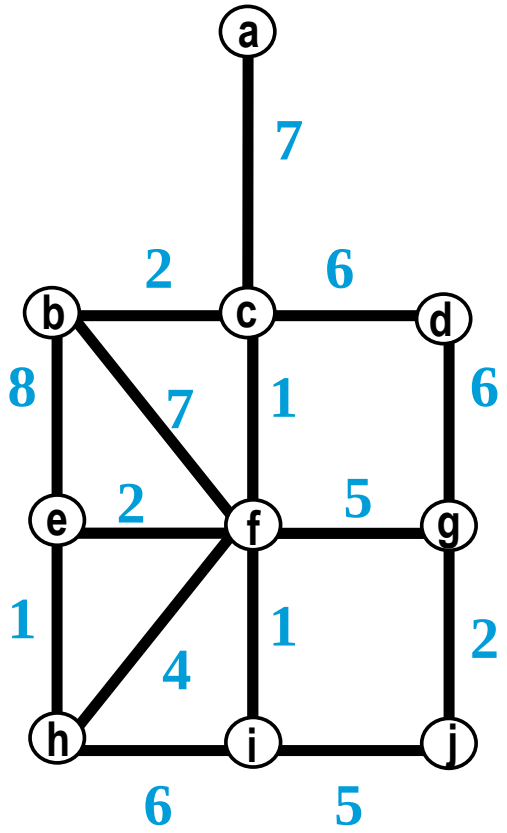
~~(c,f)~~, ~~(e,h)~~, ~~(f,i)~~, ~~(b,c)~~, ~~(e,f)~~, ~~(g,j)~~, ~~(f,h)~~, ~~(f,g)~~, ~~(i,j)~~, (c,d), (d,g), (h,i), (a,c), (b,f), (b,e)

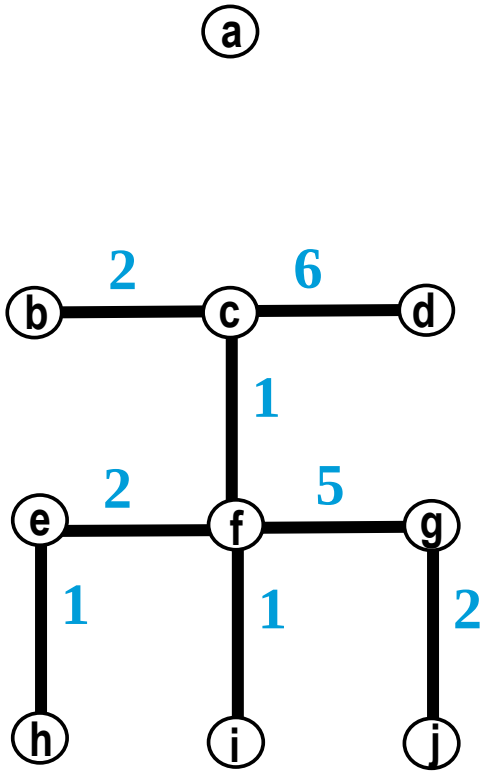




Algoritmo de Kruskal

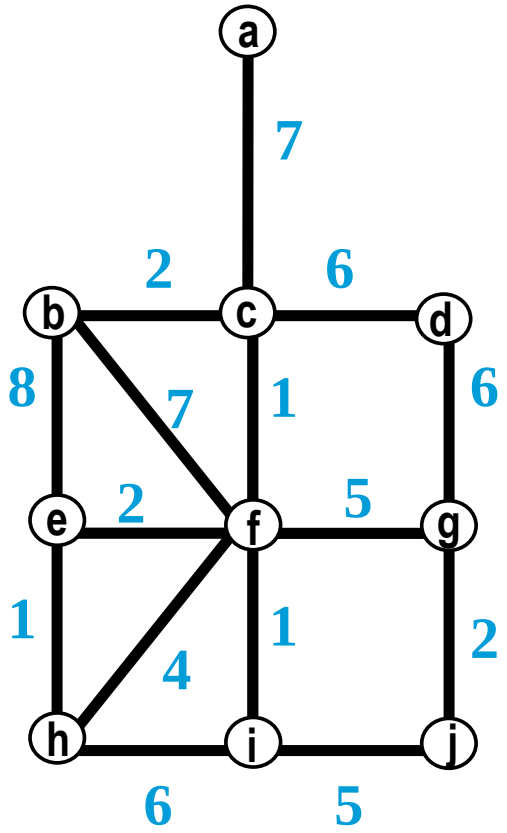
~~(c,f)~~, ~~(e,h)~~, ~~(f,i)~~, ~~(b,c)~~, ~~(e,f)~~, ~~(g,j)~~, ~~(f,h)~~, ~~(f,g)~~, ~~(i,j)~~, ~~(c,d)~~, (d,g), (h,i), (a,c), (b,f), (b,e)

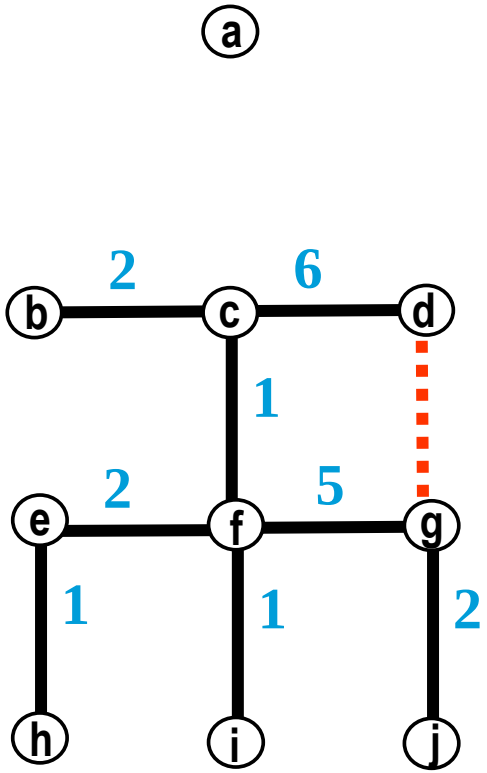




Algoritmo de Kruskal

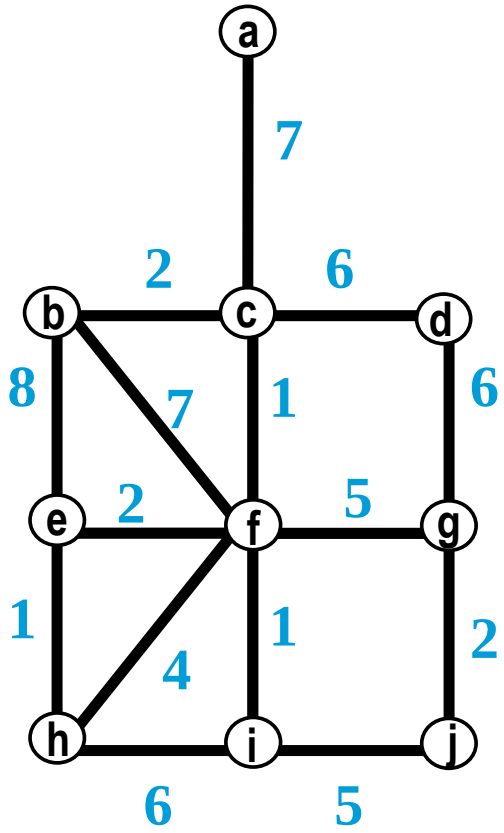
~~(c,f)~~, ~~(e,h)~~, ~~(f,i)~~, ~~(b,c)~~, ~~(e,f)~~, ~~(g,j)~~, ~~(f,h)~~, ~~(f,g)~~, ~~(i,j)~~, ~~(c,d)~~, ~~(d,g)~~, (h,i), (a,c), (b,f), (b,e)

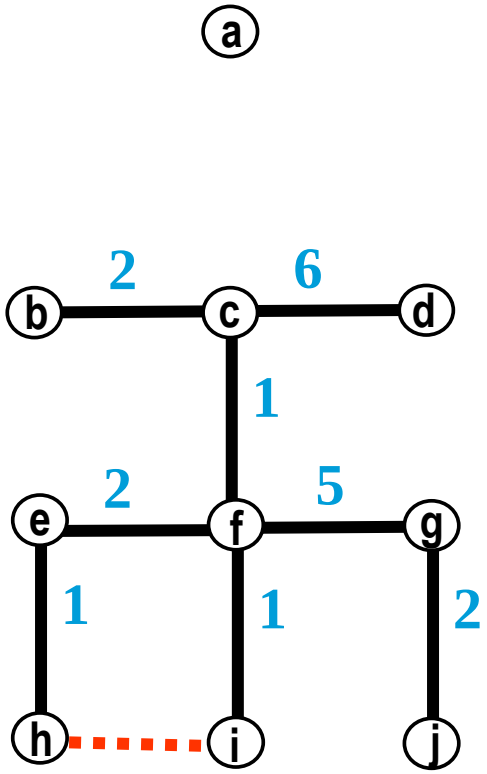




Algoritmo de Kruskal

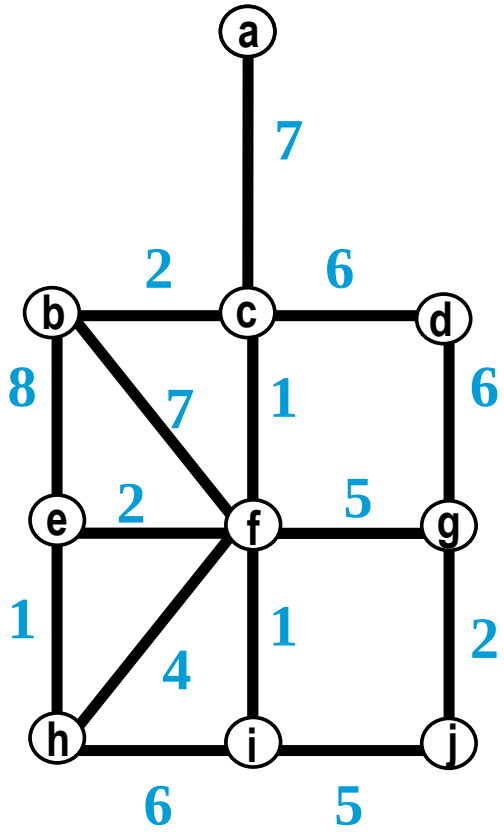
~~(c,f)~~, ~~(e,h)~~, ~~(f,i)~~, ~~(b,c)~~, ~~(e,f)~~, ~~(g,j)~~, ~~(f,h)~~, ~~(f,g)~~, ~~(i,j)~~, ~~(c,d)~~, ~~(d,g)~~, ~~(h,i)~~, (a,c), (b,f), (b,e)

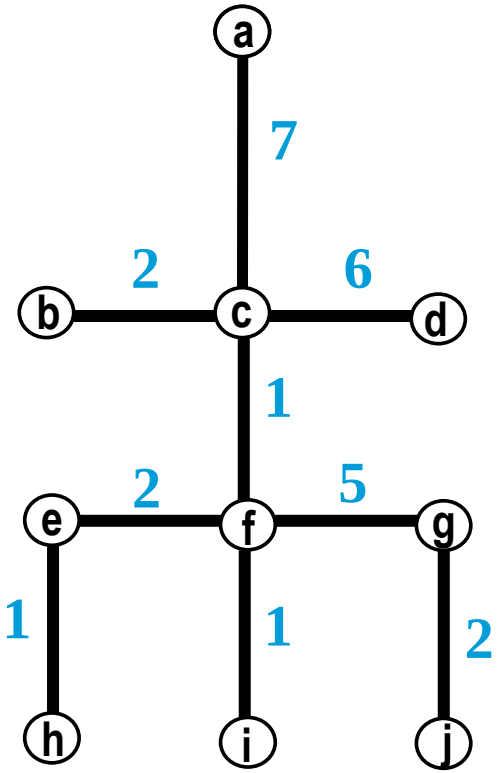




Algoritmo de Kruskal

(c,f), (e,h), (f,i), (b,c), (e,f), (g,j), (f,h), (f,g), (i,j), (c,d), (d,g), (h,i), (a,c), (b,f), (b,e)





Algoritmo de Kruskal

Este algoritmo constrói uma árvore? Se sim, a árvore é mínima?

Dados $G=(N,M)$, $D=[d_{ij}]$ a matriz distância de G.
Ordene as arestas de G em ordem não decrescente das distâncias d_{ij} no vetor $H = (h_i)$, $i = 1,2,...,m$

$T \leftarrow \{h_1\}$
$i \leftarrow 2$
Enquanto $|T| < n-1$ **faça**
    Se $T \cup h_i$ é um grafo acíclico **então**
        $T \leftarrow T \cup \{h_i\}$
        $i \leftarrow i + 1$
    Caso Contrário
        $i \leftarrow i + 1$
Escrever T {arestas da árvore geradora mínima}

???

Algoritmo de Kruskal

Prova da correção do algoritmo

Este algoritmo constrói uma árvore, pois adiciona *n*-1 arestas sem formar ciclo.

Dados $G=(N,M)$, $D=[d_{ij}]$ a matriz distância de G.
Ordene as arestas de G em ordem não decrescente das distâncias d_{ij} no vetor $H = (h_i)$, $i = 1,2,...,m$
$T \leftarrow \{h_1\}$
$i \leftarrow 2$
Enquanto $|T| < n-1$ **faça**
  Se $T \cup h_i$ é um grafo acíclico **então**
    $T \leftarrow T \cup \{h_i\}$
    $i \leftarrow i + 1$
  Caso Contrário
    $i \leftarrow i + 1$
Escrever T {arestas da árvore geradora mínima}

G é uma árvore
se e somente se
G é acíclico e possui n-1 arestas

Algoritmo de Kruskal

Prova da correção do algoritmo

Suponha que a árvore não é mínima.
Então, a adição de alguma aresta (*i*,*j*) fez com que a árvore mínima, T_{min}, não fosse obtida.
Uma vez que o algoritmo não cria ciclos, *i* e *j* estavam em componentes diferentes quando a aresta foi inserida.
Então, alguma aresta no caminho entre *i* e *j* não foi incluída pelo algoritmo, a qual produziria a árvore T_{min}.
Essa aresta teria um custo menor que (*i*,*j*). Então, o algoritmo teria desconsiderado esta aresta de peso menor.

ABSURDO.

Algoritmo de Kruskal

Qual é a complexidade?

Dados $G=(N,M)$, $D=[d_{ij}]$ a matriz distância de G.
Ordene as arestas de G em ordem não decrescente das distâncias d_{ij} no vetor $H = (h_i)$, $i = 1,2,...,m$

$T \leftarrow \{h_1\}$

$i \leftarrow 2$

Enquanto $|T| < n-1$ **faça**

  Se $T \cup h_i$ é um grafo acíclico **então**

    $T \leftarrow T \cup \{h_i\}$

    $i \leftarrow i + 1$

  Caso Contrário

    $i \leftarrow i + 1$

Escrever T {arestas da árvore geradora mínima}

Algoritmo de Kruskal

Qual é a complexidade?

Dados $G=(N,M)$, $D=[d_{ij}]$ a matriz distância de G.
Ordene as arestas de G em ordem não decrescente das distâncias d_{ij} no vetor $H = (h_i)$, $i = 1,2,...,m$
$T \leftarrow \{h_1\}$
$i \leftarrow 2$
Enquanto $|T| < n-1$ **faça**
    Se $T \cup h_i$ é um grafo acíclico **então**
        $T \leftarrow T \cup \{h_i\}$
        $i \leftarrow i + 1$
    Caso Contrário
        $i \leftarrow i + 1$
Escrever T {arestas da árvore geradora mínima}



Algoritmo de Kruskal

Complexidade (sem estruturas de dados especiais)

Ordenação das arestas: $O(m \log m) = O(m \log n)$

O "enquanto" faz $O(m)$ iterações.

Verificação de ciclo: $O(?)$

Dados $G=(N,M)$, $D=[d_{ij}]$ a matriz distância de G.
Ordene as arestas de G em ordem não decrescente das distâncias d_{ij} no vetor $H = (h_i)$, $i = 1,2,...,m$
$T \leftarrow \{h_1\}$
$i \leftarrow 2$
Enquanto $|T| < n-1$ **faça**
  Se $T \cup h_i$ é um grafo acíclico **então**
    $T \leftarrow T \cup \{h_i\}$
    $i \leftarrow i + 1$
  Caso Contrário
    $i \leftarrow i + 1$
Escrever T {arestas da árvore geradora mínima}

Algoritmo de Kruskal

Complexidade (sem estruturas de dados especiais)

Verificação de ciclo: utilizando um vetor para controlar as diferentes componentes.

***O*(1) para ver se os vértices pertencem a mesma componente**
***O*(*n*) para atualizar a estrutura (é atualizada *n* vezes)**

$$O(n^2)$$

Algoritmo de Kruskal

Complexidade (sem estruturas de dados especiais)

$O(m \log n) + O(m - n + n^2)$

Final: $O(m \log n)$

Obs: A verificação de ciclo pode ser implementada em $O(n \log n)$ utilizando dois vetores

Algoritmo de Kruskal

Complexidade

Pesquisa: Qual a complexidade do algoritmo usando estruturas de dados mais sofisticadas?



Dica: Verificar o uso de "*conjuntos disjuntos*" (*union-find*).

Árvore Geradora Mínima

Como Prim (1957) pensou?

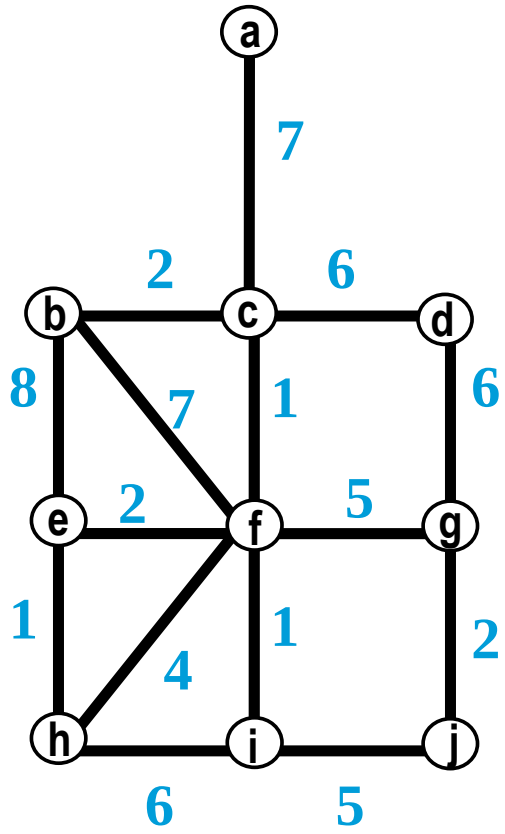
Escolhe um vértice qualquer.
Inclui a menor aresta incidente a um vértice da árvore, tal que o outro terminal da aresta não esteja na árvore.
Parar quando todos os vértices estiverem na árvore.

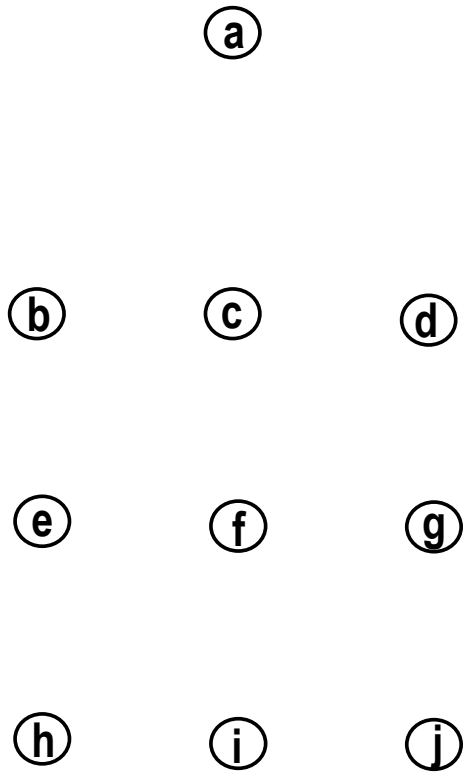
Prim, R. C. Shortest connection networks and some generalizations. *Bell System Technical Journal*, v. 36, pp. 1389-1401, 1957.

Algoritmo de Prim

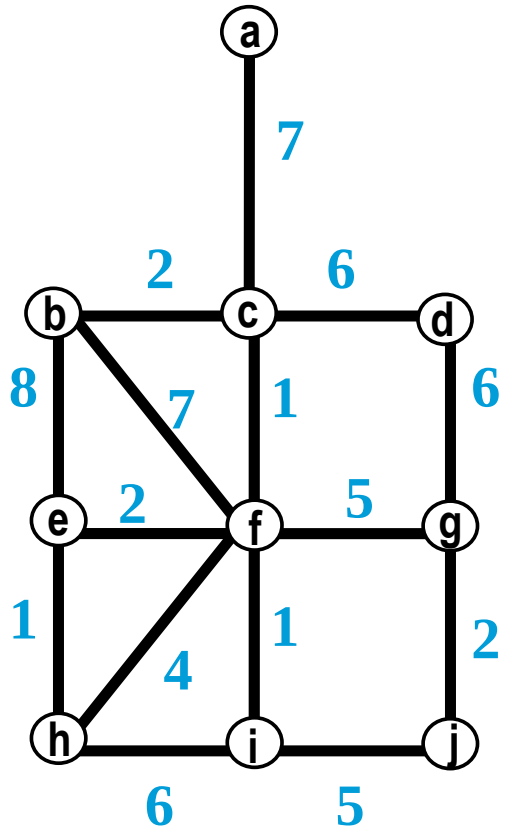
Ler $G=(N,M)$ e $D=[d_{ij}]$ a matriz distância de G
Escolha qualquer vértice $i \in N$
$Z \leftarrow \{i\}$
$V \leftarrow N \setminus \{i\}$
$T_{min} \leftarrow \emptyset$
Enquanto $Z \neq N$ **Faça**
    Encontrar a aresta $(j,k) \in M$ tal que $j \in Z$, $k \in V$
    e d_{jk} é mínimo
        $Z \leftarrow Z \cup \{k\}$
        $V \leftarrow V \setminus \{k\}$
        $\boldsymbol{T_{Min}} \leftarrow \boldsymbol{T_{Min}} \cup (j,k)$
Escrever $\boldsymbol{T_{Min}}$ {o conjunto das arestas da árvore geradora mínima}

Algoritmo de Prim



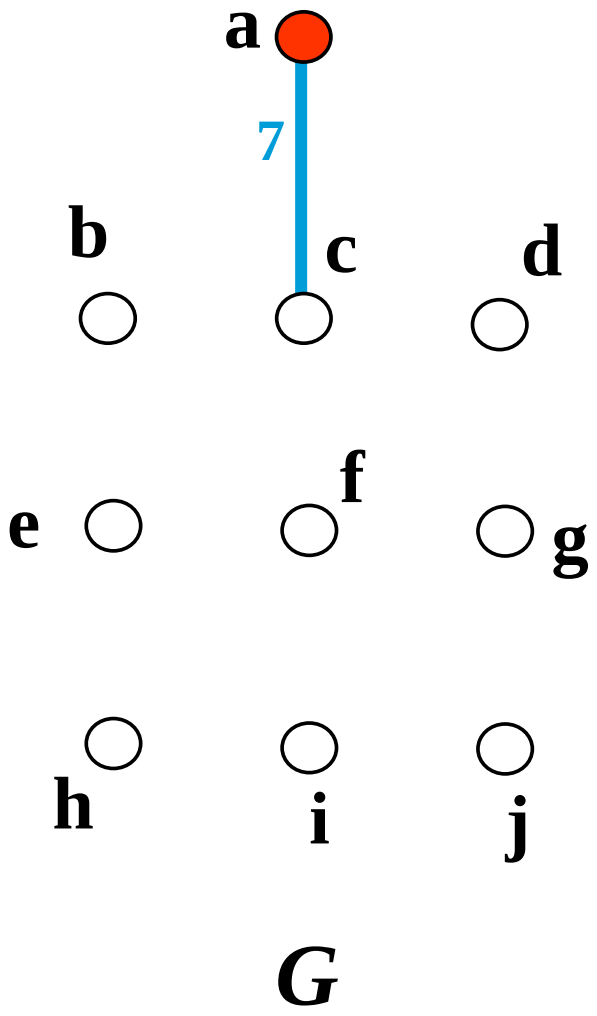


Algoritmo de Prim



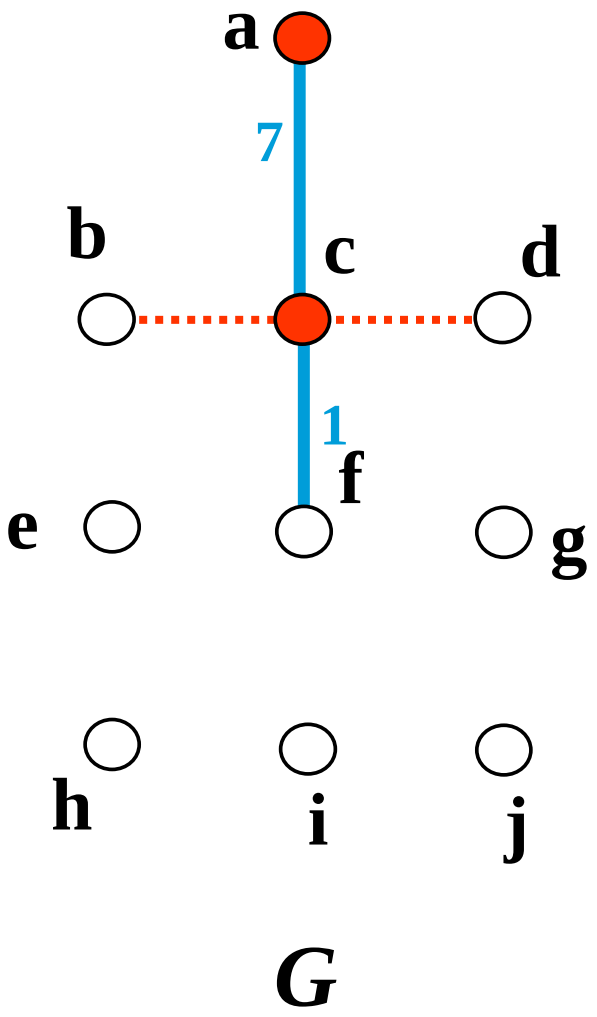
	a	b	c	d	e	f	g	h	i	j
a	∞	∞	7	∞	∞	∞	∞	∞	∞	∞
b	∞	∞	2	∞	8	7	∞	∞	∞	∞
c	7	2	∞	6	∞	1	∞	∞	∞	∞
d	∞	∞	6	∞	∞	∞	6	∞	∞	∞
e	∞	8	∞	∞	∞	2	∞	1	∞	∞
f	∞	7	1	∞	2	∞	5	4	1	∞
g	∞	∞	∞	6	∞	5	∞	∞	∞	2
h	∞	∞	∞	∞	1	4	∞	∞	6	∞
i	∞	∞	∞	∞	∞	1	∞	6	∞	5
j	∞	∞	∞	∞	∞	∞	2	∞	5	∞

Algoritmo de Prim



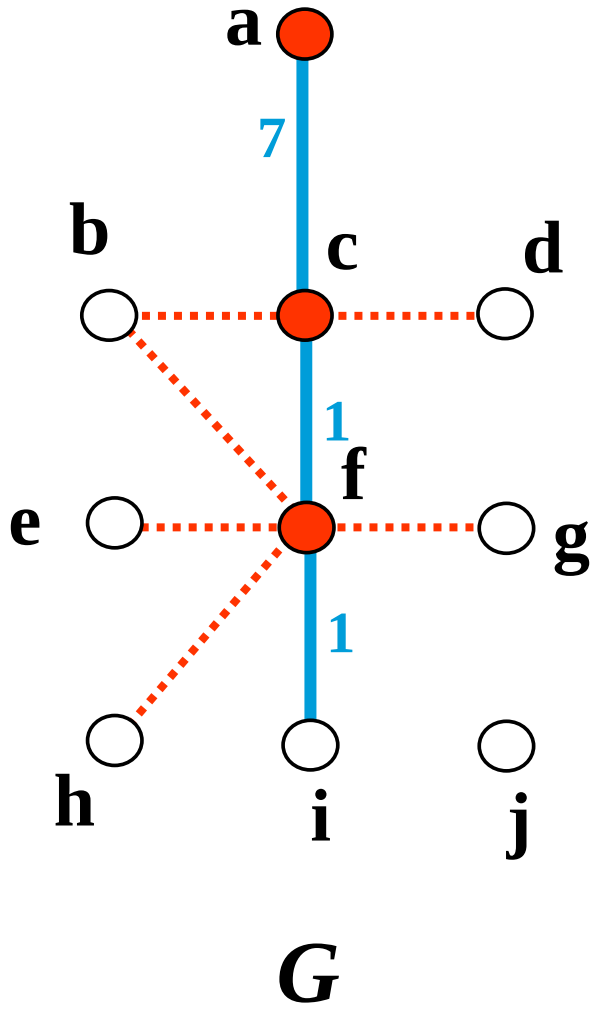
	a	b	c	d	e	f	g	h	i	j
a	∞	∞	7	∞	∞	∞	∞	∞	∞	∞
b	∞	∞	2	∞	8	7	∞	∞	∞	∞
c	7	2	∞	6	∞	1	∞	∞	∞	∞
d	∞	∞	6	∞	∞	∞	6	∞	∞	∞
e	∞	8	∞	∞	∞	2	∞	1	∞	∞
f	∞	7	1	∞	2	∞	5	4	1	∞
g	∞	∞	∞	6	∞	5	∞	∞	∞	2
h	∞	∞	∞	∞	1	4	∞	∞	6	∞
i	∞	∞	∞	∞	∞	1	∞	6	∞	5
j	∞	∞	∞	∞	∞	∞	2	∞	5	∞

Algoritmo de Prim



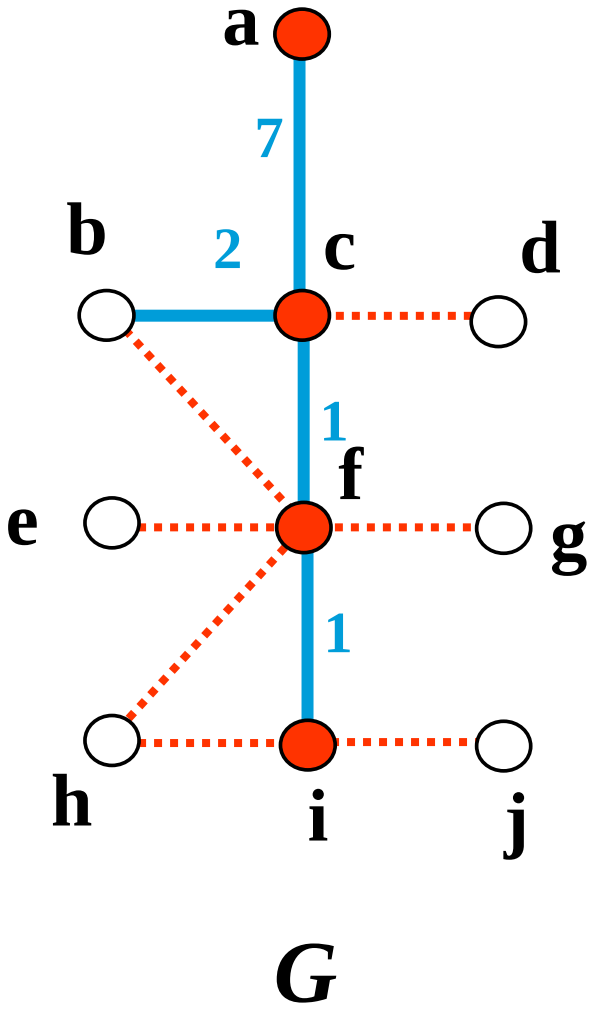
	a	b	c	d	e	f	g	h	i	j
a	∞	∞	7	∞	∞	∞	∞	∞	∞	∞
b	∞	∞	2	∞	8	7	∞	∞	∞	∞
c	7	2	∞	6	∞	1	∞	∞	∞	∞
d	∞	∞	6	∞	∞	∞	6	∞	∞	∞
e	∞	8	∞	∞	∞	2	∞	1	∞	∞
f	∞	7	1	∞	2	∞	5	4	1	∞
g	∞	∞	∞	6	∞	5	∞	∞	∞	2
h	∞	∞	∞	∞	1	4	∞	∞	6	∞
i	∞	∞	∞	∞	∞	1	∞	6	∞	5
j	∞	∞	∞	∞	∞	∞	2	∞	5	∞

Algoritmo de Prim



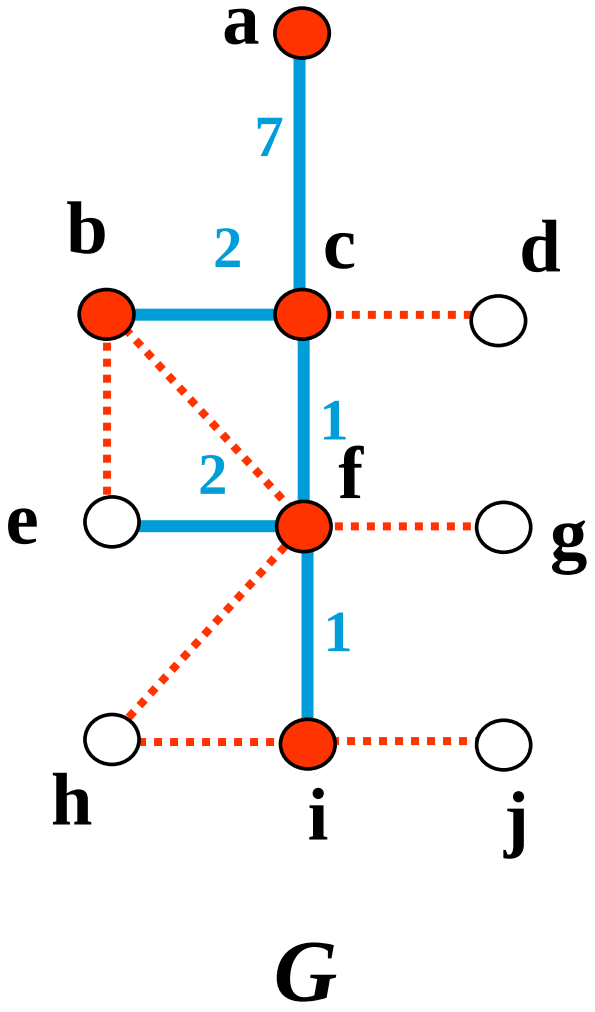
	a	b	c	d	e	f	g	h	i	j
a	∞	∞	7	∞	∞	∞	∞	∞	∞	∞
b	∞	∞	2	∞	8	7	∞	∞	∞	∞
c	7	2	∞	6	∞	1	∞	∞	∞	∞
d	∞	∞	6	∞	∞	∞	6	∞	∞	∞
e	∞	8	∞	∞	∞	2	∞	1	∞	∞
f	∞	7	1	∞	2	∞	5	4	1	∞
g	∞	∞	∞	6	∞	5	∞	∞	∞	2
h	∞	∞	∞	∞	1	4	∞	∞	6	∞
i	∞	∞	∞	∞	∞	1	∞	6	∞	5
j	∞	∞	∞	∞	∞	∞	2	∞	5	∞

Algoritmo de Prim



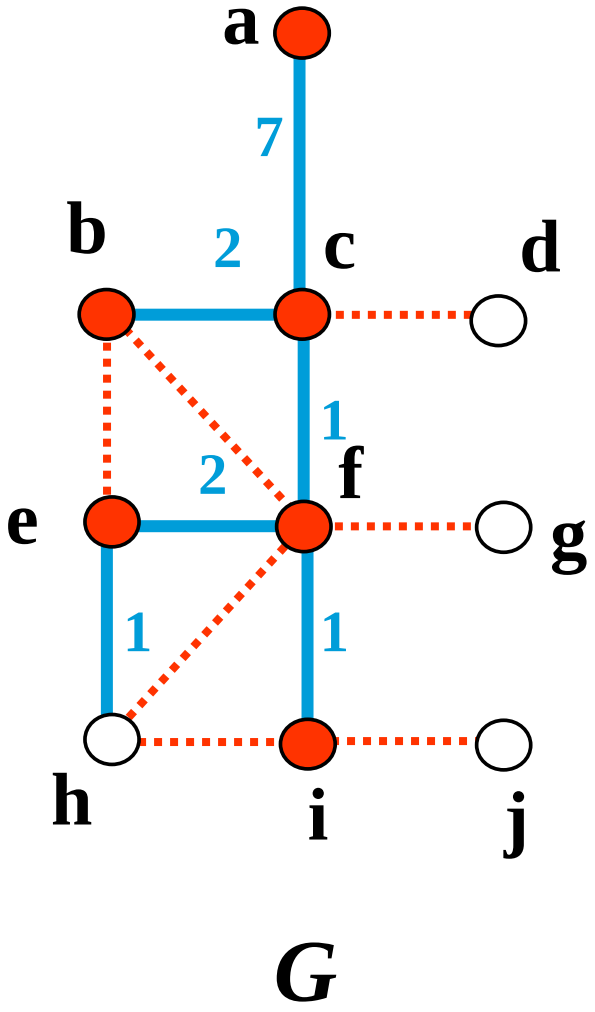
	a	b	c	d	e	f	g	h	i	j
a	∞	∞	7	∞	∞	∞	∞	∞	∞	∞
b	∞	∞	2	∞	8	7	∞	∞	∞	∞
c	7	2	∞	6	∞	1	∞	∞	∞	∞
d	∞	∞	6	∞	∞	∞	6	∞	∞	∞
e	∞	8	∞	∞	∞	2	∞	1	∞	∞
f	∞	7	1	∞	2	∞	5	4	1	∞
g	∞	∞	∞	6	∞	5	∞	∞	∞	2
h	∞	∞	∞	∞	1	4	∞	∞	6	∞
i	∞	∞	∞	∞	∞	1	∞	6	∞	5
j	∞	∞	∞	∞	∞	∞	2	∞	5	∞

Algoritmo de Prim



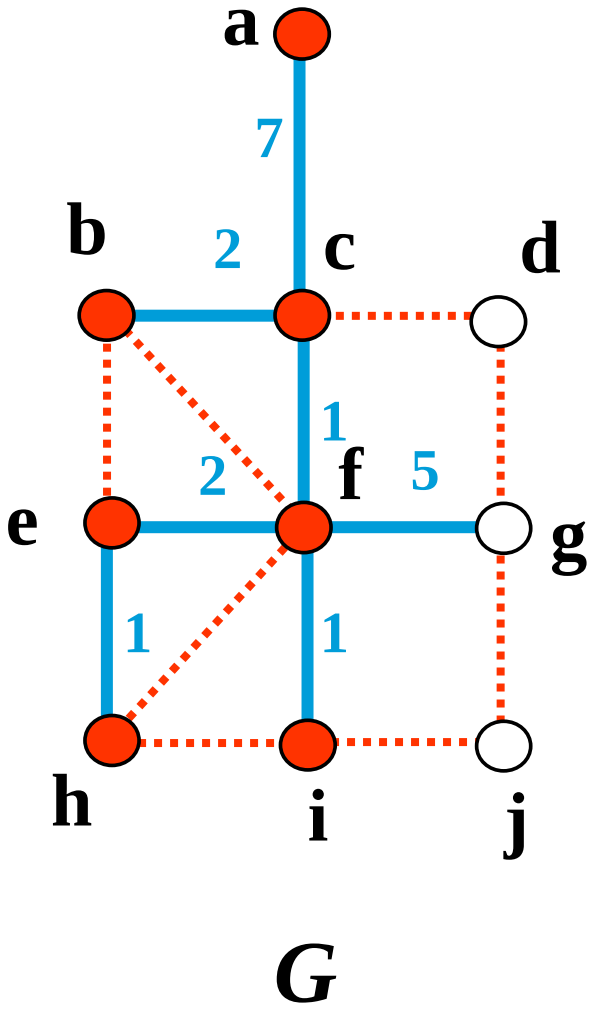
	a	b	c	d	e	f	g	h	i	j
a	∞	∞	7	∞	∞	∞	∞	∞	∞	∞
b	∞	∞	2	∞	8	7	∞	∞	∞	∞
c	7	2	∞	6	∞	1	∞	∞	∞	∞
d	∞	∞	6	∞	∞	∞	6	∞	∞	∞
e	∞	8	∞	∞	∞	2	∞	1	∞	∞
f	∞	7	1	∞	2	∞	5	4	1	∞
g	∞	∞	∞	6	∞	5	∞	∞	∞	2
h	∞	∞	∞	∞	1	4	∞	∞	6	∞
i	∞	∞	∞	∞	∞	1	∞	6	∞	5
j	∞	∞	∞	∞	∞	∞	2	∞	5	∞

Algoritmo de Prim



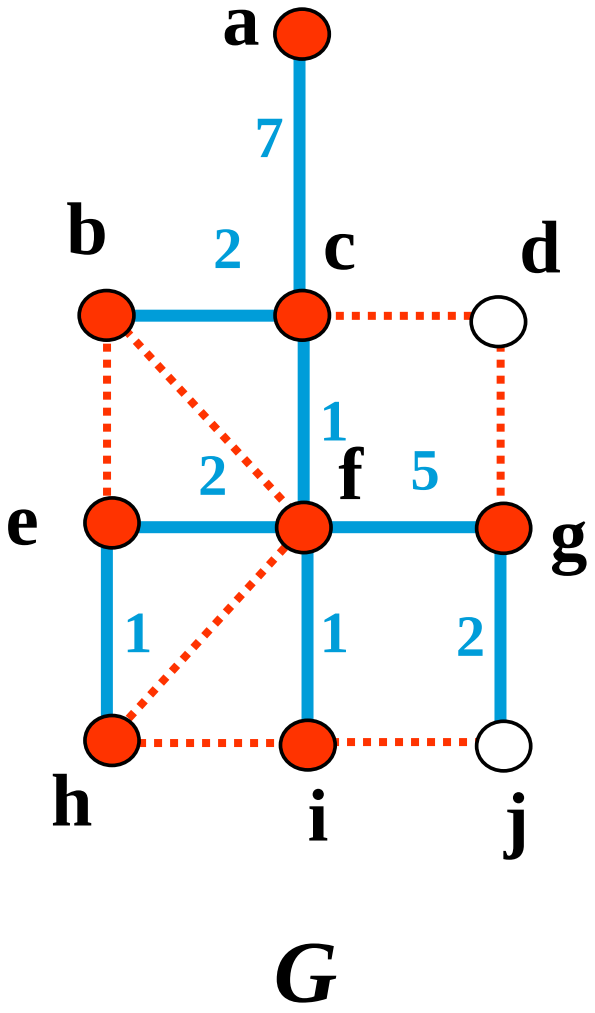
	a	b	c	d	e	f	g	h	i	j
a	∞	∞	7	∞	∞	∞	∞	∞	∞	∞
b	∞	∞	2	∞	8	7	∞	∞	∞	∞
c	7	2	∞	6	∞	1	∞	∞	∞	∞
d	∞	∞	6	∞	∞	∞	6	∞	∞	∞
e	∞	8	∞	∞	∞	2	∞	1	∞	∞
f	∞	7	1	∞	2	∞	5	4	1	∞
g	∞	∞	∞	6	∞	5	∞	∞	∞	2
h	∞	∞	∞	∞	1	4	∞	∞	6	∞
i	∞	∞	∞	∞	∞	1	∞	6	∞	5
j	∞	∞	∞	∞	∞	∞	2	∞	5	∞

Algoritmo de Prim



	a	b	c	d	e	f	g	h	i	j
a	∞	∞	7	∞	∞	∞	∞	∞	∞	∞
b	∞	∞	2	∞	8	7	∞	∞	∞	∞
c	7	2	∞	6	∞	1	∞	∞	∞	∞
d	∞	∞	6	∞	∞	∞	6	∞	∞	∞
e	∞	8	∞	∞	∞	2	∞	1	∞	∞
f	∞	7	1	∞	2	∞	5	4	1	∞
g	∞	∞	∞	6	∞	5	∞	∞	∞	2
h	∞	∞	∞	∞	1	4	∞	∞	6	∞
i	∞	∞	∞	∞	∞	1	∞	6	∞	5
j	∞	∞	∞	∞	∞	∞	2	∞	5	∞

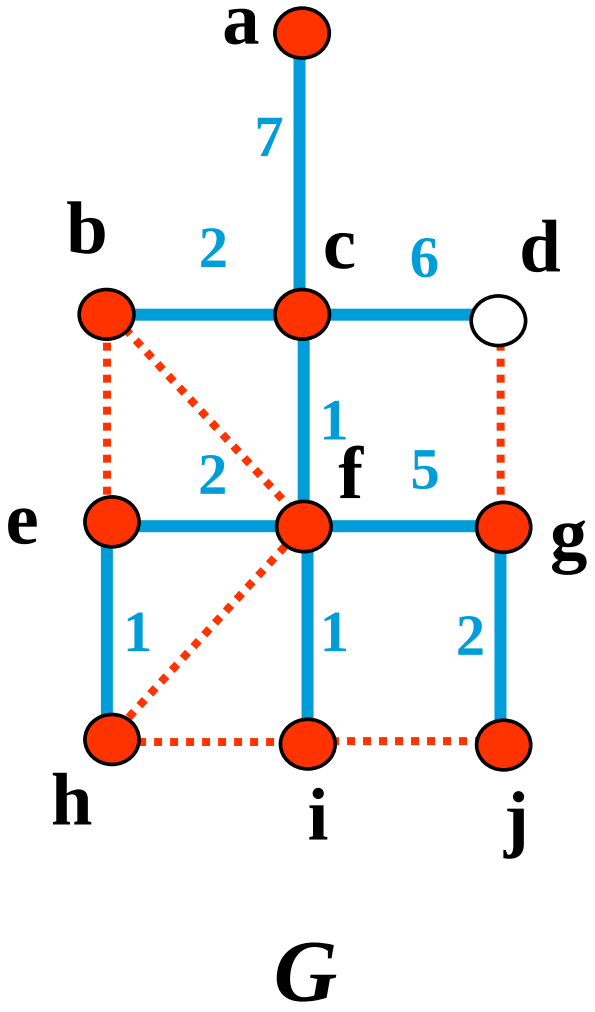
Algoritmo de Prim



G

	a	b	c	d	e	f	g	h	i	j
a	∞	∞	7	∞	∞	∞	∞	∞	∞	∞
b	∞	∞	2	∞	8	7	∞	∞	∞	∞
c	7	2	∞	6	∞	1	∞	∞	∞	∞
d	∞	∞	6	∞	∞	∞	6	∞	∞	∞
e	∞	8	∞	∞	∞	2	∞	1	∞	∞
f	∞	7	1	∞	2	∞	5	4	1	∞
g	∞	∞	∞	6	∞	5	∞	∞	∞	2
h	∞	∞	∞	∞	1	4	∞	∞	6	∞
i	∞	∞	∞	∞	∞	1	∞	6	∞	5
j	∞	∞	∞	∞	∞	∞	2	∞	5	∞

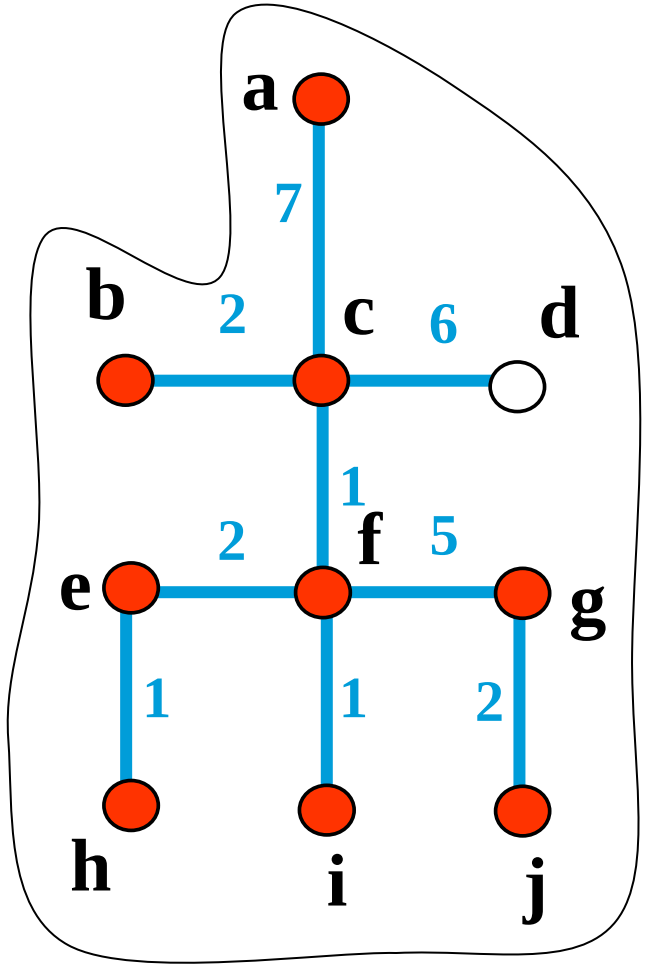
Algoritmo de Prim



G

	a	b	c	d	e	f	g	h	i	j
a	∞	∞	7	∞	∞	∞	∞	∞	∞	∞
b	∞	∞	2	∞	8	7	∞	∞	∞	∞
c	7	2	∞	6	∞	1	∞	∞	∞	∞
d	∞	∞	6	∞	∞	∞	6	∞	∞	∞
e	∞	8	∞	∞	∞	2	∞	1	∞	∞
f	∞	7	1	∞	2	∞	5	4	1	∞
g	∞	∞	∞	6	∞	5	∞	∞	∞	2
h	∞	∞	∞	∞	1	4	∞	∞	6	∞
i	∞	∞	∞	∞	∞	1	∞	6	∞	5
j	∞	∞	∞	∞	∞	∞	2	∞	5	∞

Algoritmo de Prim



T

Custo = 27

Algoritmo de Prim

Exercício

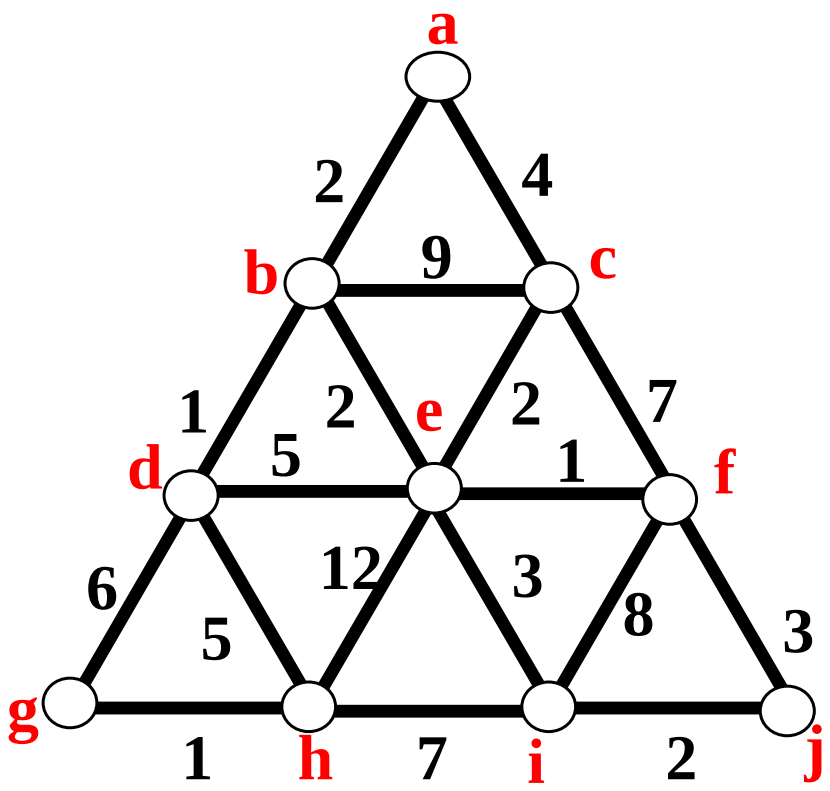
O algoritmo de Prim produz uma árvore geradora?

Neste caso, ela é mínima?

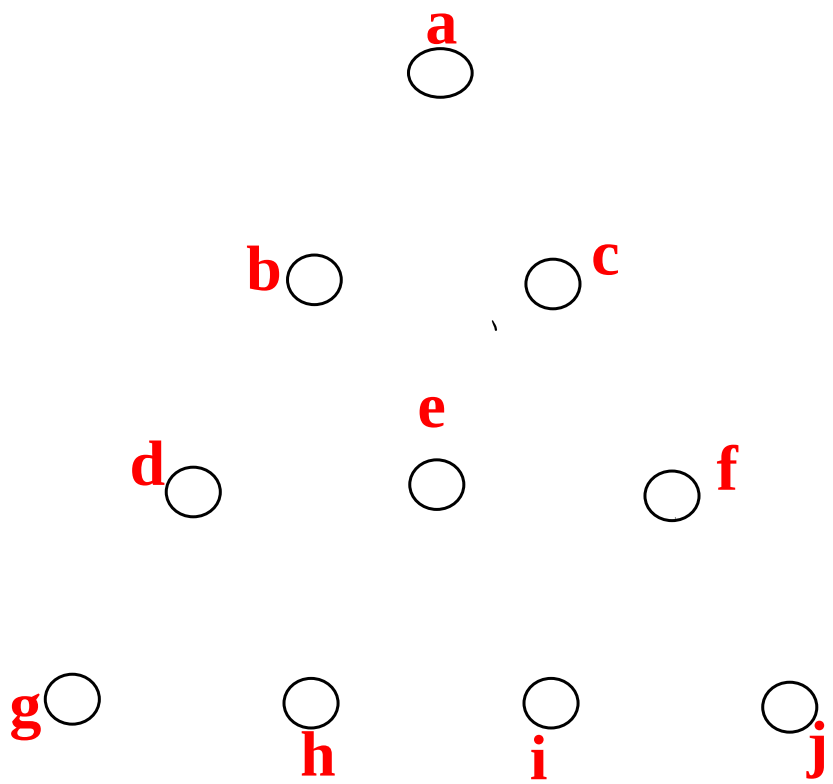
Ler $G=(N,M)$ e $D=[d_{ij}]$ a matriz distância de G
Escolha qualquer vértice $i \in N$
$Z \leftarrow \{i\}$
$V \leftarrow N \setminus \{i\}$
$T_{min} \leftarrow \emptyset$
Enquanto $Z \neq N$ **Faça**
  Encontrar a aresta $(j,k) \in M$ tal que $j \in Z$, $k \in V$ e d_{jk} é mínimo
    $Z \leftarrow Z \cup \{k\}$
    $V \leftarrow V \setminus \{k\}$
    $T_{Min} \leftarrow T_{Min} \cup (j,k)$
Escrever T_{Min}

Algoritmo de Prim

Exercício Prim



G



Algoritmo de Prim

Complexidade
(Prova – Cormen et al., 2002)

O "enquanto" faz *n*-1 iterações, *O*(*n*).

Encontrar a aresta $(j,k) \in M$ tal que $j \in Z$, $k \in V$ e d_{jk} é mínimo

Complexidade: (?)

Ler $G=(N,M)$ e $D=[d_{ij}]$ a matriz distância de G
Escolha qualquer vértice $i \in N$
$Z \leftarrow \{i\}$
$V \leftarrow N \setminus \{i\}$
$T_{min} \leftarrow \emptyset$
Enquanto $Z \neq N$ **Faça**
  Encontrar a aresta $(j,k) \in M$ tal que $j \in Z$, $k \in V$ e d_{jk} é mínimo
    $Z \leftarrow Z \cup \{k\}$
    $V \leftarrow V \setminus \{k\}$
    $T_{Min} \leftarrow T_{Min} \cup (j,k)$
Escrever T_{Min}

Algoritmo de Prim

Complexidade

Utilizar uma lista de prioridades para os vértices que não estão na árvore de modo que:

Chave(*v*) = peso da menor aresta que liga o vértice *v* a um vértice que já está na árvore

Operações: remoção do primeiro elemento + atualização das chaves

Algoritmo de Prim

Complexidade

Remoção do primeiro elemento da lista de prioridades = $O(\log n)$

Número de remoções = número de iterações do enquanto

Total desta etapa = $O(n \log n)$

Algoritmo de Prim

Complexidade

Atualização de uma chave na lista de prioridades = $O(\log n)$

A atualização é realizada $O(n^2)$ vezes durante toda a execução do algoritmo (para um grafo completo).

No caso de um grafo não completo a atualização é realizada $O(m)$ vezes, pois a soma dos comprimentos de todas as listas de adjacência é $2m$.

Algoritmo de Prim

Complexidade

Final: $O(n \log n + m \log n) = O(m \log n)$

Utilizando Heap de Fibonacci é possível implementar em $O(m + n \log n) = O(m)$

Árvore Geradora Mínima

Como Borůvka (1926) pensou?



Fonte: Wikipedia

Forma uma floresta com cada vértice sendo uma árvore.
Liga cada árvore com outra pela aresta de peso mínimo.
Parar quando for uma árvore.

*Borůvka, Otakar (1926), "O jistém problému minimálním", Práce Moravské přírodovědecké společnosti, **3** (3): 37–58*

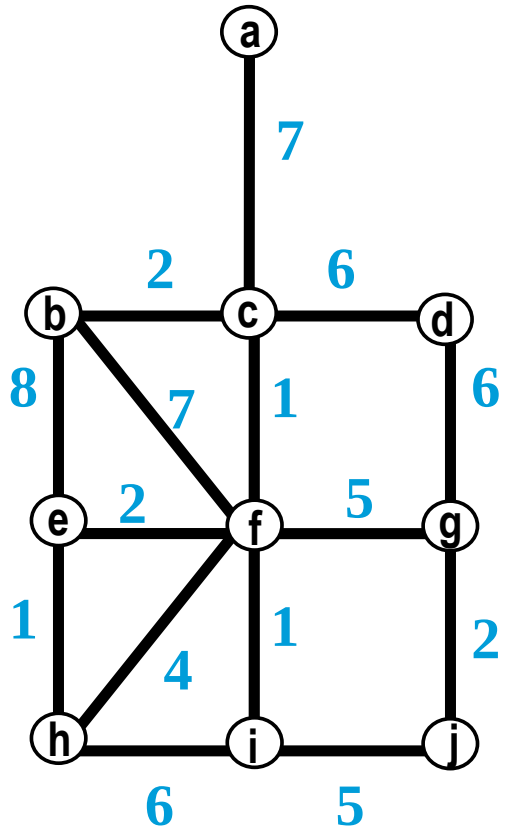
Algoritmo de Borůvka

Ler $G=(N,M)$ e $D=[d_{ij}]$ a matriz distância de G
F_0 é uma floresta inicial com n subárvores T_j , $j=1,..,n$
$i \leftarrow 0$
Enquanto F_i não for uma árvore **Faça**
  Para cada $T_j \in F_i$
    Determine a menor aresta (x_α, y_α) incidente em T_j onde $x_\alpha \in T_j$ e $y_\alpha \notin T_j$

  Faça $F_{i+1} \leftarrow F_i \cup \left[\bigcup_\alpha (x_\alpha, y_\alpha) \right]$

  $i \leftarrow i+1$
Escrever F_i {arestas da árvore geradora mínima}

Algoritmo de Borůvka



a

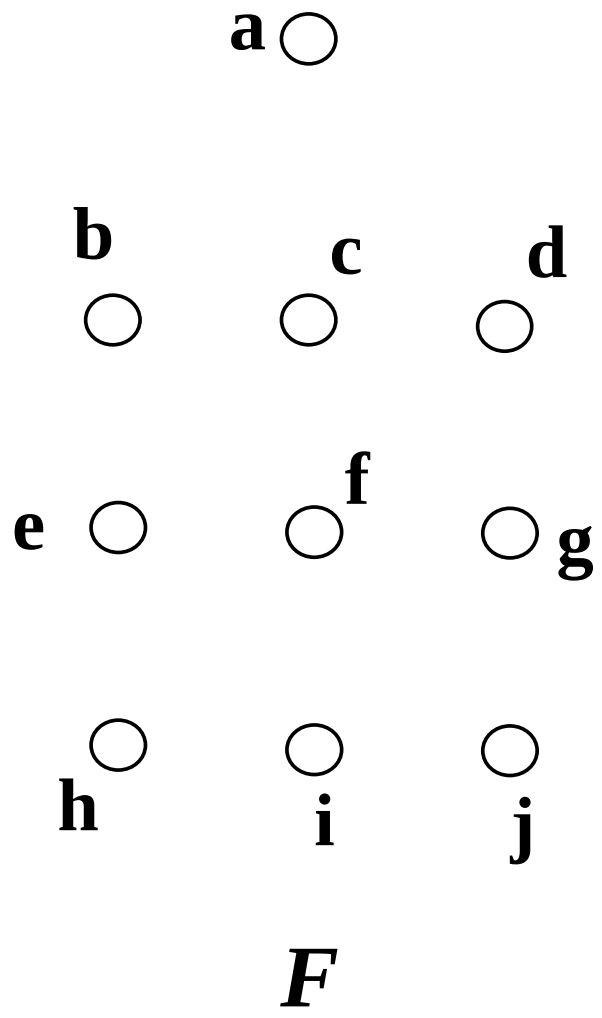
b c d

e f g

h i j

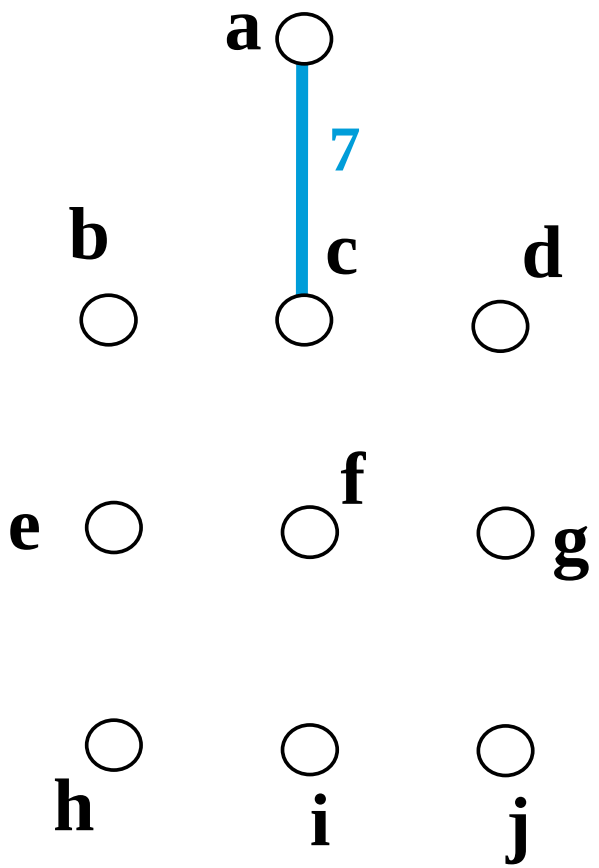
Algoritmo de Borůvka

	a	b	c	d	e	f	g	h	i	j
a	∞	∞	7	∞	∞	∞	∞	∞	∞	∞
b	∞	∞	2	∞	8	7	∞	∞	∞	∞
c	7	2	∞	6	∞	1	∞	∞	∞	∞
d	∞	∞	6	∞	∞	∞	6	∞	∞	∞
e	∞	8	∞	∞	∞	2	∞	1	∞	∞
f	∞	7	1	∞	2	∞	5	4	1	∞
g	∞	∞	∞	6	∞	5	∞	∞	∞	2
h	∞	∞	∞	∞	1	4	∞	∞	6	∞
i	∞	∞	∞	∞	∞	1	∞	6	∞	5
j	∞	∞	∞	∞	∞	∞	2	∞	5	∞



Algoritmo de Borůvka

	a	b	c	d	e	f	g	h	i	j
a	∞	∞	7	∞	∞	∞	∞	∞	∞	∞
b	∞	∞	2	∞	8	7	∞	∞	∞	∞
c	7	2	∞	6	∞	1	∞	∞	∞	∞
d	∞	∞	6	∞	∞	∞	6	∞	∞	∞
e	∞	8	∞	∞	∞	2	∞	1	∞	∞
f	∞	7	1	∞	2	∞	5	4	1	∞
g	∞	∞	∞	6	∞	5	∞	∞	∞	2
h	∞	∞	∞	∞	1	4	∞	∞	6	∞
i	∞	∞	∞	∞	∞	1	∞	6	∞	5
j	∞	∞	∞	∞	∞	∞	2	∞	5	∞

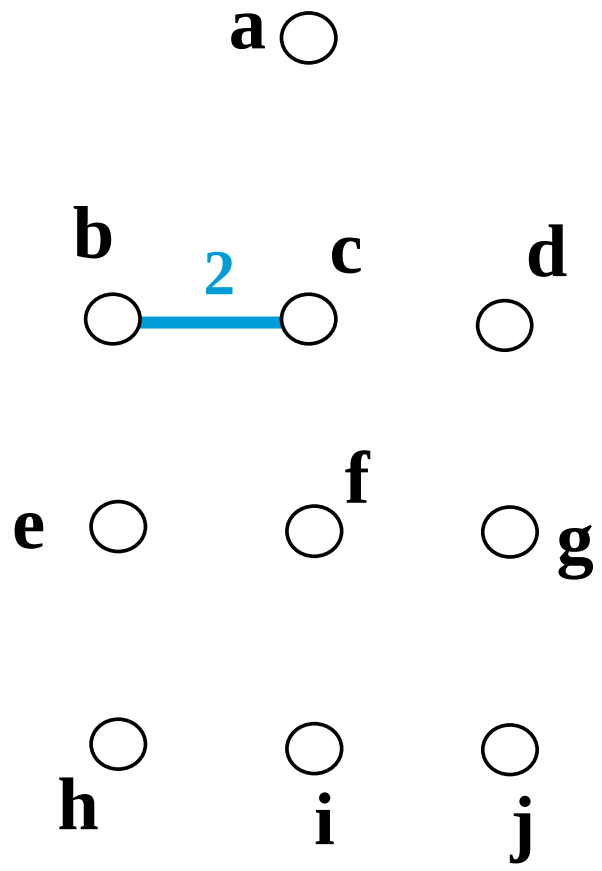


Vértice a

F_a

Algoritmo de Borůvka

	a	b	c	d	e	f	g	h	i	j
a	∞	∞	7	∞	∞	∞	∞	∞	∞	∞
b	∞	∞	2	∞	8	7	∞	∞	∞	∞
c	7	2	∞	6	∞	1	∞	∞	∞	∞
d	∞	∞	6	∞	∞	∞	6	∞	∞	∞
e	∞	8	∞	∞	∞	2	∞	1	∞	∞
f	∞	7	1	∞	2	∞	5	4	1	∞
g	∞	∞	∞	6	∞	5	∞	∞	∞	2
h	∞	∞	∞	∞	1	4	∞	∞	6	∞
i	∞	∞	∞	∞	∞	1	∞	6	∞	5
j	∞	∞	∞	∞	∞	∞	2	∞	5	∞

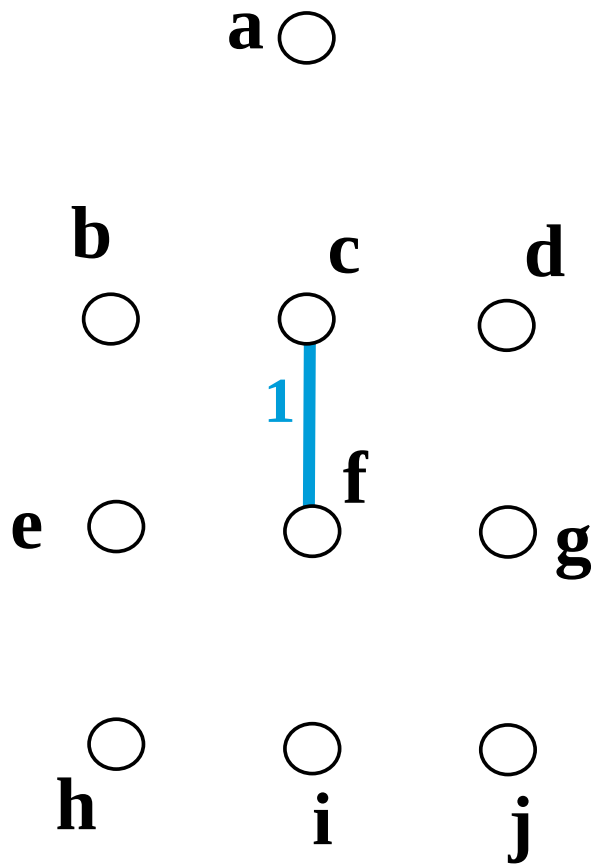


Vértice b

F_b

Algoritmo de Borůvka

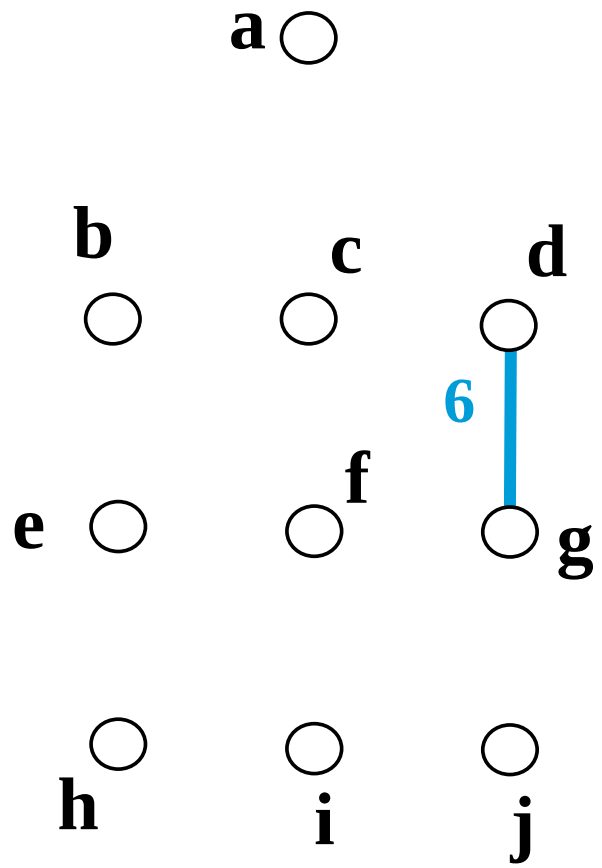
	a	b	c	d	e	f	g	h	i	j
a	∞	∞	7	∞	∞	∞	∞	∞	∞	∞
b	∞	∞	2	∞	8	7	∞	∞	∞	∞
c	7	2	∞	6	∞	1	∞	∞	∞	∞
d	∞	∞	6	∞	∞	∞	6	∞	∞	∞
e	∞	8	∞	∞	∞	2	∞	1	∞	∞
f	∞	7	1	∞	2	∞	5	4	1	∞
g	∞	∞	∞	6	∞	5	∞	∞	∞	2
h	∞	∞	∞	∞	1	4	∞	∞	6	∞
i	∞	∞	∞	∞	∞	1	∞	6	∞	5
j	∞	∞	∞	∞	∞	∞	2	∞	5	∞



Vértice c

Algoritmo de Borůvka

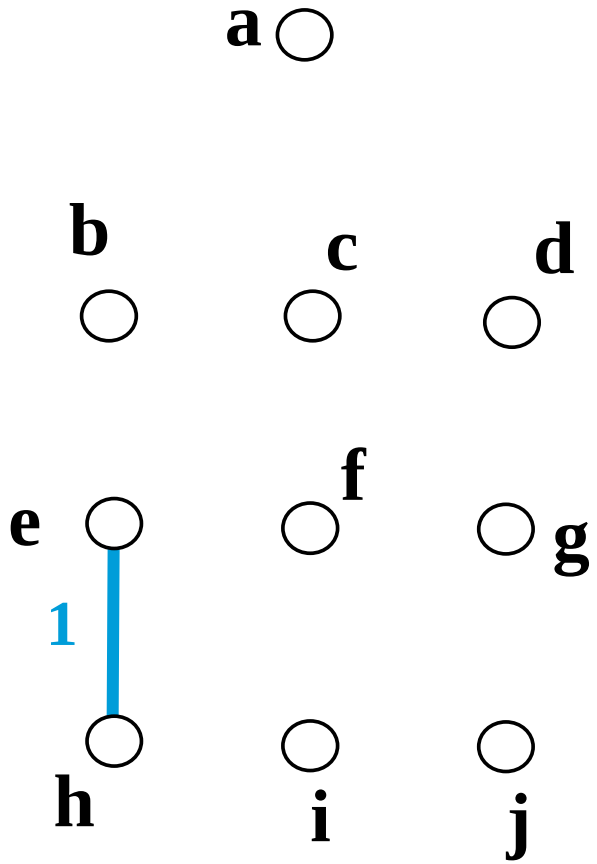
	a	b	c	d	e	f	g	h	i	j
a	∞	∞	7	∞	∞	∞	∞	∞	∞	∞
b	∞	∞	2	∞	8	7	∞	∞	∞	∞
c	7	2	∞	6	∞	1	∞	∞	∞	∞
d	∞	∞	6	∞	∞	∞	6	∞	∞	∞
e	∞	8	∞	∞	∞	2	∞	1	∞	∞
f	∞	7	1	∞	2	∞	5	4	1	∞
g	∞	∞	∞	6	∞	5	∞	∞	∞	2
h	∞	∞	∞	∞	1	4	∞	∞	6	∞
i	∞	∞	∞	∞	∞	1	∞	6	∞	5
j	∞	∞	∞	∞	∞	∞	2	∞	5	∞



Vértice d

Algoritmo de Borůvka

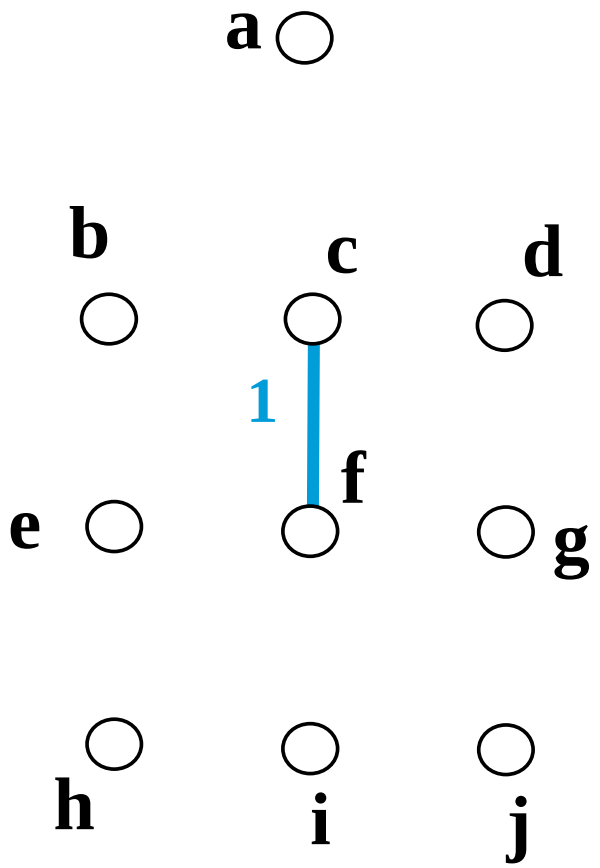
	a	b	c	d	e	f	g	h	i	j
a	∞	∞	7	∞	∞	∞	∞	∞	∞	∞
b	∞	∞	2	∞	8	7	∞	∞	∞	∞
c	7	2	∞	6	∞	1	∞	∞	∞	∞
d	∞	∞	6	∞	∞	∞	6	∞	∞	∞
e	∞	8	∞	∞	∞	2	∞	1	∞	∞
f	∞	7	1	∞	2	∞	5	4	1	∞
g	∞	∞	∞	6	∞	5	∞	∞	∞	2
h	∞	∞	∞	∞	1	4	∞	∞	6	∞
i	∞	∞	∞	∞	∞	1	∞	6	∞	5
j	∞	∞	∞	∞	∞	∞	2	∞	5	∞



Vértice e

Algoritmo de Borůvka

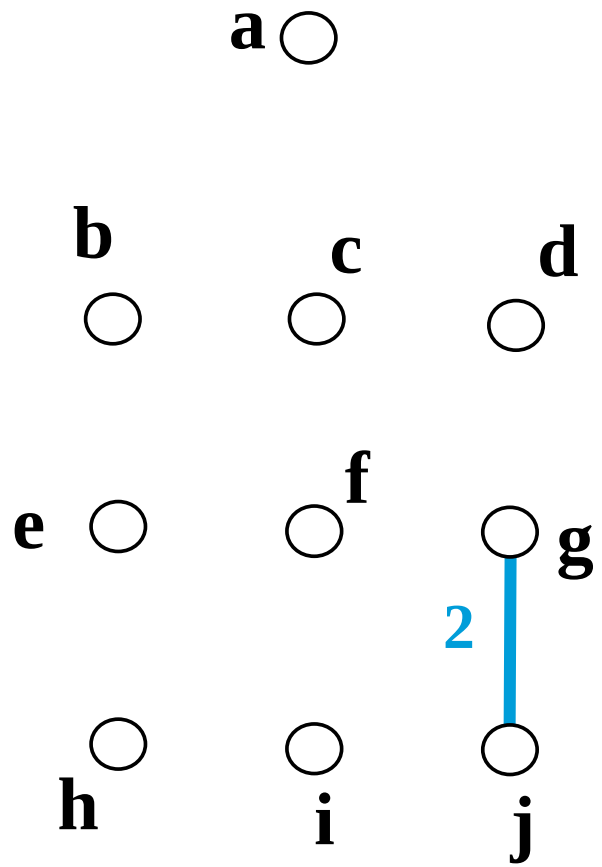
	a	b	c	d	e	f	g	h	i	j
a	∞	∞	7	∞	∞	∞	∞	∞	∞	∞
b	∞	∞	2	∞	8	7	∞	∞	∞	∞
c	7	2	∞	6	∞	1	∞	∞	∞	∞
d	∞	∞	6	∞	∞	∞	6	∞	∞	∞
e	∞	8	∞	∞	∞	2	∞	1	∞	∞
f	∞	7	1	∞	2	∞	5	4	1	∞
g	∞	∞	∞	6	∞	5	∞	∞	∞	2
h	∞	∞	∞	∞	1	4	∞	∞	6	∞
i	∞	∞	∞	∞	∞	1	∞	6	∞	5
j	∞	∞	∞	∞	∞	∞	2	∞	5	∞



Vértice f

Algoritmo de Borůvka

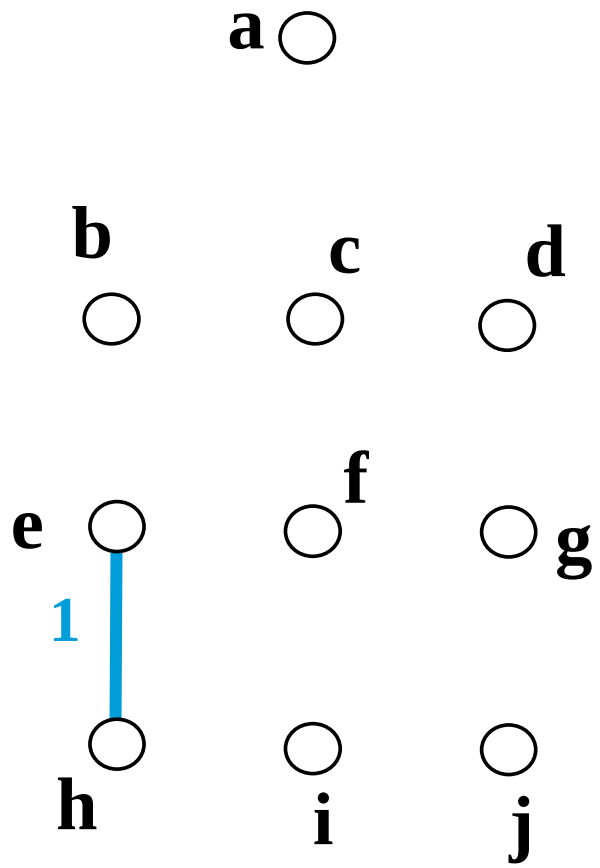
	a	b	c	d	e	f	g	h	i	j
a	∞	∞	7	∞	∞	∞	∞	∞	∞	∞
b	∞	∞	2	∞	8	7	∞	∞	∞	∞
c	7	2	∞	6	∞	1	∞	∞	∞	∞
d	∞	∞	6	∞	∞	∞	6	∞	∞	∞
e	∞	8	∞	∞	∞	2	∞	1	∞	∞
f	∞	7	1	∞	2	∞	5	4	1	∞
g	∞	∞	∞	6	∞	5	∞	∞	∞	2
h	∞	∞	∞	∞	1	4	∞	∞	6	∞
i	∞	∞	∞	∞	∞	1	∞	6	∞	5
j	∞	∞	∞	∞	∞	∞	2	∞	5	∞



Vértice g

Algoritmo de Borůvka

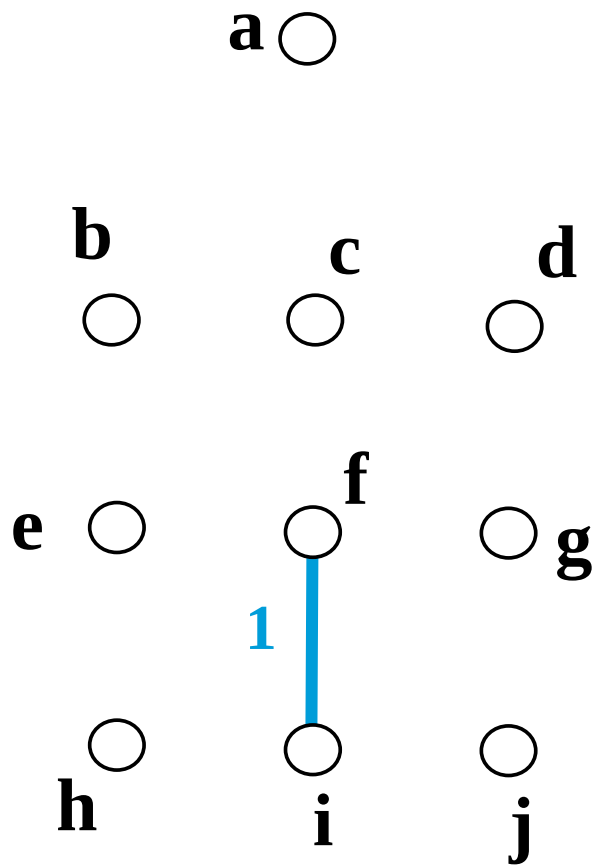
	a	b	c	d	e	f	g	h	i	j
a	∞	∞	7	∞	∞	∞	∞	∞	∞	∞
b	∞	∞	2	∞	8	7	∞	∞	∞	∞
c	7	2	∞	6	∞	1	∞	∞	∞	∞
d	∞	∞	6	∞	∞	∞	6	∞	∞	∞
e	∞	8	∞	∞	∞	2	∞	1	∞	∞
f	∞	7	1	∞	2	∞	5	4	1	∞
g	∞	∞	∞	6	∞	5	∞	∞	∞	2
h	∞	∞	∞	∞	1	4	∞	∞	6	∞
i	∞	∞	∞	∞	∞	1	∞	6	∞	5
j	∞	∞	∞	∞	∞	∞	2	∞	5	∞



Vértice h

Algoritmo de Borůvka

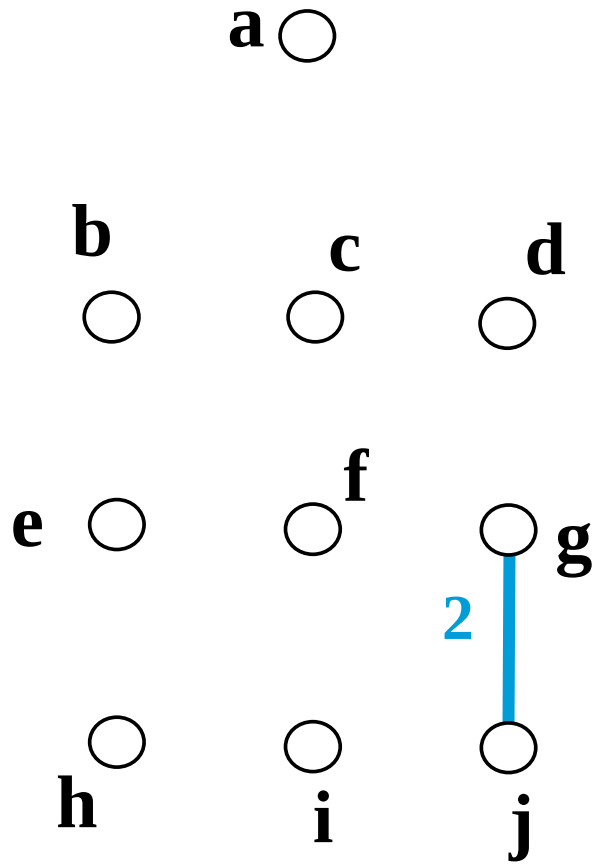
	a	b	c	d	e	f	g	h	i	j
a	∞	∞	7	∞	∞	∞	∞	∞	∞	∞
b	∞	∞	2	∞	8	7	∞	∞	∞	∞
c	7	2	∞	6	∞	1	∞	∞	∞	∞
d	∞	∞	6	∞	∞	∞	6	∞	∞	∞
e	∞	8	∞	∞	∞	2	∞	1	∞	∞
f	∞	7	1	∞	2	∞	5	4	1	∞
g	∞	∞	∞	6	∞	5	∞	∞	∞	2
h	∞	∞	∞	∞	1	4	∞	∞	6	∞
i	∞	∞	∞	∞	∞	1	∞	6	∞	5
j	∞	∞	∞	∞	∞	∞	2	∞	5	∞



Vértice i

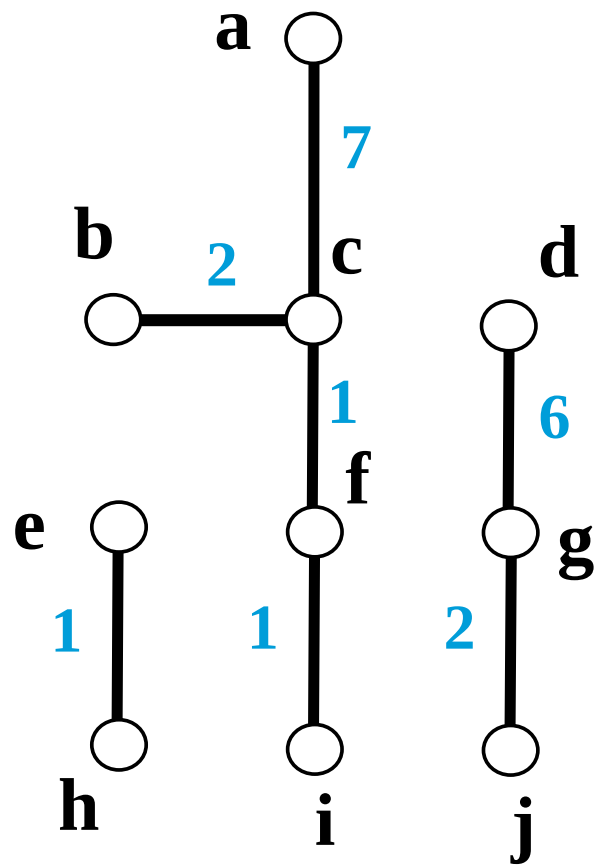
Algoritmo de Borůvka

	a	b	c	d	e	f	g	h	i	j
a	∞	∞	7	∞	∞	∞	∞	∞	∞	∞
b	∞	∞	2	∞	8	7	∞	∞	∞	∞
c	7	2	∞	6	∞	1	∞	∞	∞	∞
d	∞	∞	6	∞	∞	∞	6	∞	∞	∞
e	∞	8	∞	∞	∞	2	∞	1	∞	∞
f	∞	7	1	∞	2	∞	5	4	1	∞
g	∞	∞	∞	6	∞	5	∞	∞	∞	2
h	∞	∞	∞	∞	1	4	∞	∞	6	∞
i	∞	∞	∞	∞	∞	1	∞	6	∞	5
j	∞	∞	∞	∞	∞	∞	2	∞	5	∞



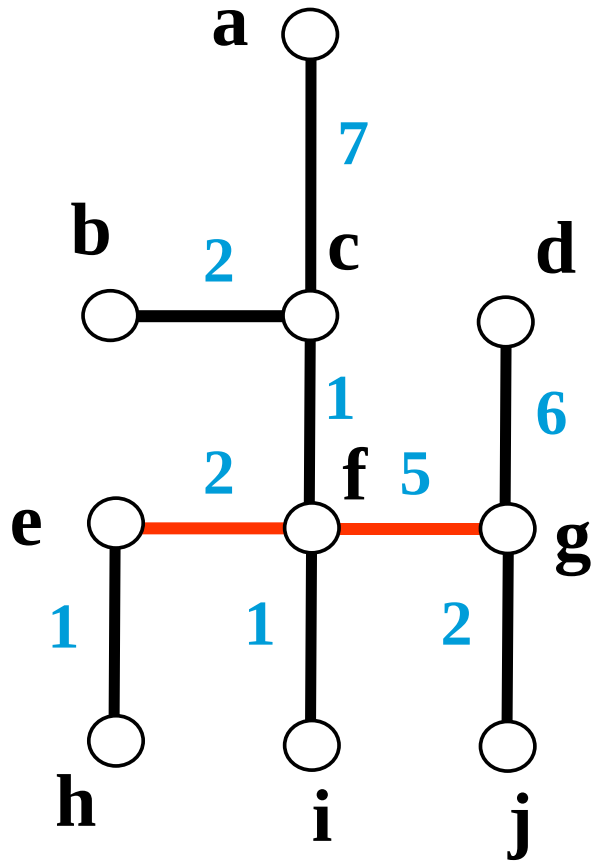
Vértice j

Algoritmo de Borůvka



Floresta da Primeira Iteração

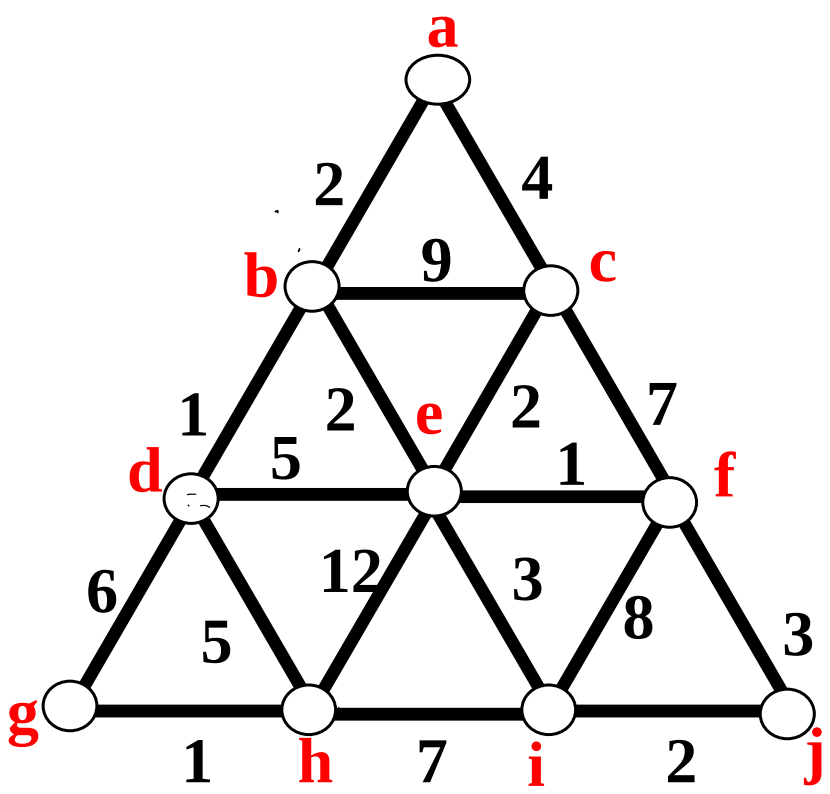
Algoritmo de Borůvka



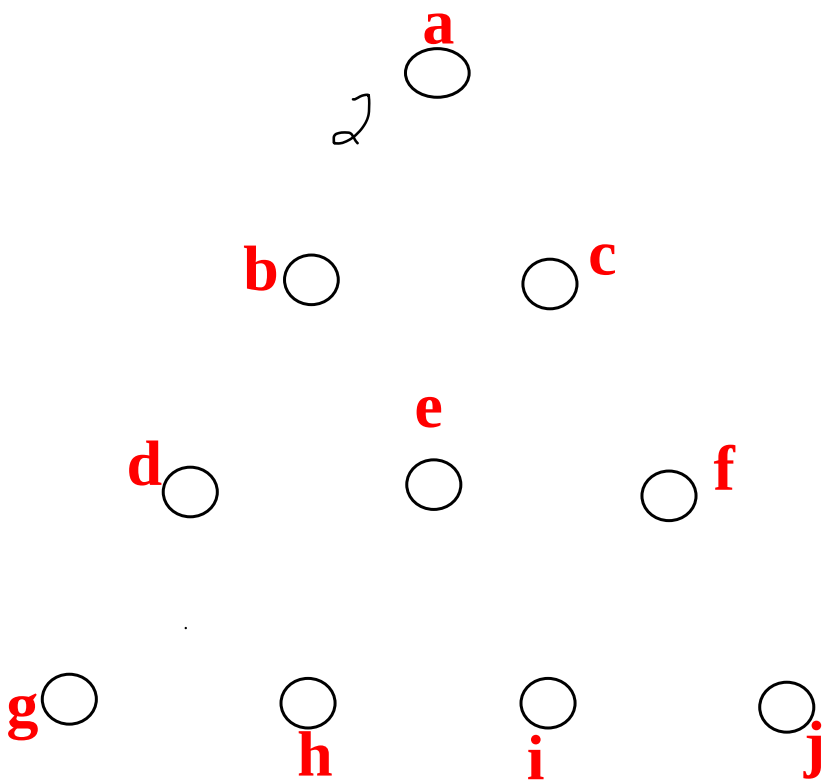
Arestas da segunda iteração

Algoritmo de Borůvka

Exercício Borůvka



G



Algoritmo de Borůvka

Como cada árvore da floresta inclui a menor aresta incidente na solução em cada iteração, se existirem arestas iguais no grafo pode ocorrer **ERRO** formando ciclo.

O algoritmo, da forma descrita só funciona corretamente se as arestas de *G* forem distintas. Para evitar que possam ocorrer inclusões indevidas por ocasião das uniões das subárvores deve-se ordenar lexicograficamente as arestas de *G*.

Exercício: Exemplificar um caso onde a aplicação do algoritmo sem a ordenação lexicográfica FALHA.

Algoritmo de Borůvka

Em cada iteração do comando enquanto, o número de árvores da floresta diminui de um fator de 2.

Portanto, são executadas $\log_2 n$ iterações do laço mais externo.

Em cada iteração são examinadas $O(m)$ arestas. Portanto, o algoritmo, sem estruturas de dados especiais pode ser implementado em $O(m \log n)$.

Árvore Geradora Máxima

Árvore Geradora Máxima

Exemplo de aplicação (Christofides, 1975): ***Redes de Comunicação***

Considere o problema de enviar informação confidencial para n pessoas.

O grafo G tem n vértices (um para cada pessoa) e existe uma aresta entre os nós i e j se é possível enviar a informação diretamente de i para j.

A probabilidade associada ao vazamento da informação entre i e j é p_{ij}.

Portanto, a probabilidade da informação ser enviada entre i e j sem que haja vazamento é $q_{ij} = 1 - p_{ij}$.

Para enviar a informação a informação entre as n pessoas deseja-se encontrar um subgrafo gerador de G para o qual o produto de q_{ij} (sobre todas as arestas) seja máximo.

Árvore Geradora Máxima

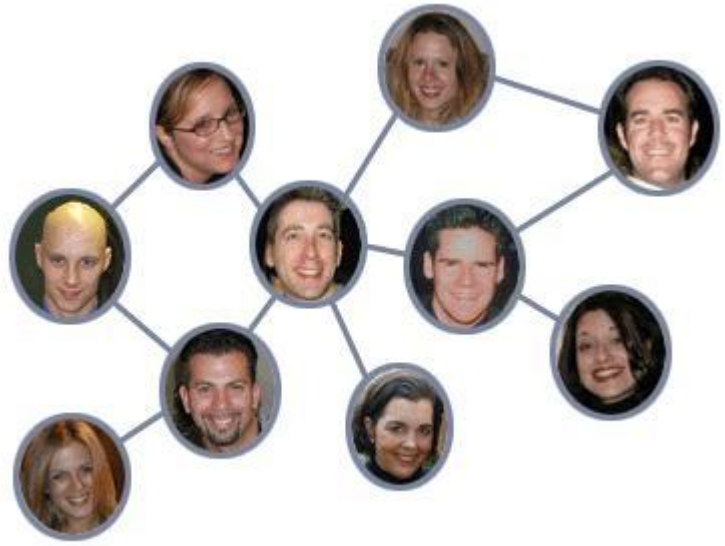
Exemplo de aplicação (Christofides, 1975): ***Redes de Comunicação***

Deve-se substituir q_{ij} por $w(i,j) = \log q_{ij,}$ portanto, reduzindo o problema a encontrar uma árvore geradora máxima.

Outro exemplo de aplicação:
Confiabilidade de redes

-Vértices = nós da rede

-p_{ij} é a probabilidade da conexão entre i e j falhar.



Árvore Geradora Máxima

Árvore Gargalo (Bottleneck)

Considere G um grafo conexo no qual cada aresta e possui capacidade $w(e)$.

Para qualquer caminho $W = v_0, v_1,...,v_p$, $e_i = (v_{i-1}, v_i)$

A capacidade de W é dada por $c(W) = \min\{w(e_i): i = 1,...,p\}$

Para cada par de vértices u e v de G queremos determinar o caminho entre u e v com máxima capacidade.

Este problema corresponde a encontrar a árvore geradora máxima de G (Hu, 1961).

Hu, T.C. (1961) The maximum capacity route problem, *Operations Research* 9, 898-900

Árvore Geradora Máxima

Árvore Gargalo (Bottleneck)

Teorema. Considere G um grafo conexo no qual cada aresta e possui capacidade $w(e)$ e T uma árvore geradora máxima de G. Então, para cada par de vértices u e v de G o único caminho entre u e v em T é um caminho de máxima capacidade em G.

Prova. Considere W o caminho entre u e v em T e e uma aresta em W tal que $w(e) = c(W)$. Suponha que existe um caminho W' entre u e v em G tal que $c(W') > c(W)$. Considere $S_T(e)$ o corte de G definido anteriormente e $E(S_T(e))$ o co-ciclo correspondente. Uma vez que u e v estão em diferentes componentes conexas de $T\backslash\{e\}$, o caminho W' deve conter alguma aresta $f \in E(S_T(e))$. Como $c(W') > c(W)$, temos $w(f) > w(e)$. Neste caso, $(T\cup\{f\})\backslash\{e\}$ seria uma árvore geradora maior que T. Contradição com o fato de T ser árvore geradora máxima.

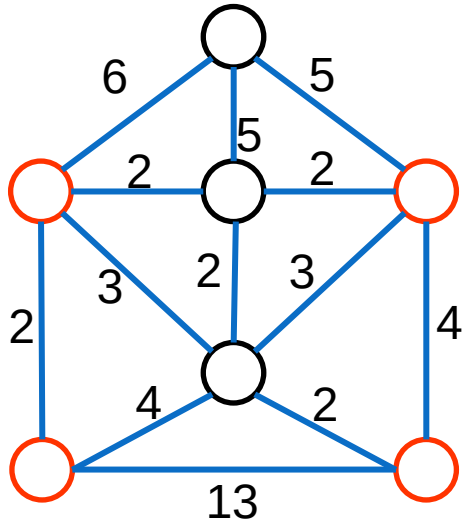
■

Árvore de Steiner

Considere $G = (V,E)$ um grafo conexo no qual cada aresta e possui peso $w(e)$, $V = R \cup S$, $R \cap S = \varnothing$. O problema consiste em encontrar uma árvore T de custo mínimo, que contenha os vértices de R, chamados de *vértices terminais*.

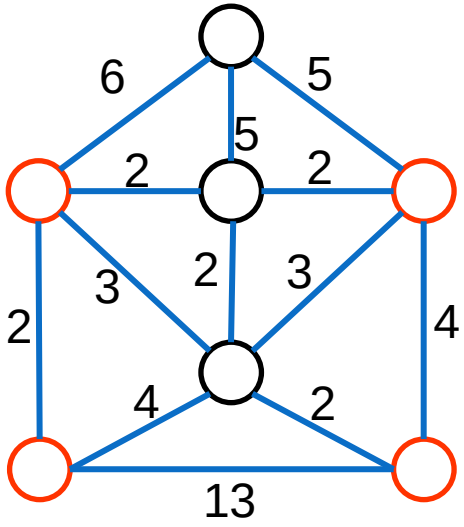
Os vértices de S podem estar ou não na árvore. Tais vértices são chamados de vértices de Steiner.

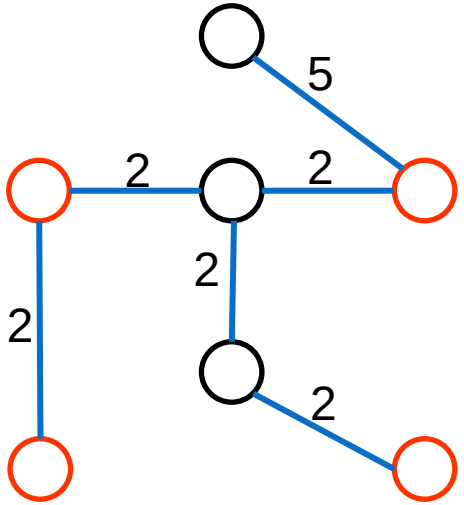
Problema NP-difícil.



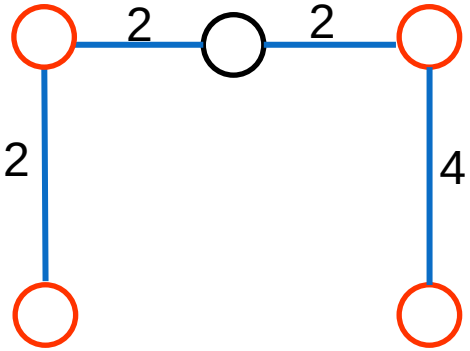
Terminais

Árvore de Steiner





Árvore Geradora Mínima

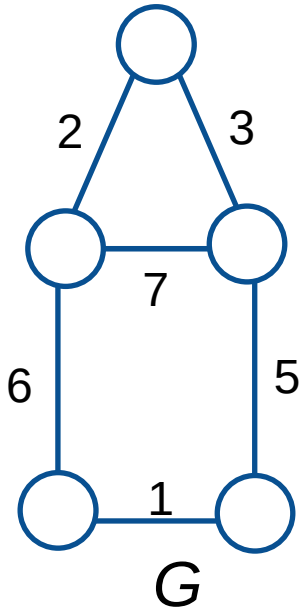


Árvore de Steiner

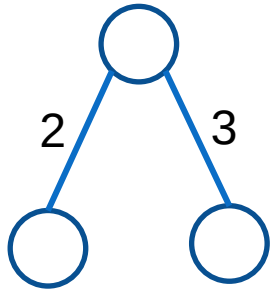
k- Árvore Geradora Mínima

Considere $G = (V,E)$, $|V| = n$, um grafo conexo no qual cada aresta e possui peso $w(e)$ e k uma constante, tal que $k < n$.

O problema consiste em encontrar a árvore geradora mínima de G com k vértices.



G



3-AGM

Ravi, R., Sundaram, R., Marathe, M. V., Rosenkrantz, D. J., & Ravi, S. S. (1996). Spanning trees—short or small. *SIAM Journal on Discrete Mathematics*, *9*(2), 178-200.

Árvores Geradoras Mínimas

Trabalho com revisão de algoritmos para árvores geradoras mínimas

•BAZLAMAÇCI, C. F.; HINDI, K. S. Minimum-weight spanning tree algorithms: a survey and empirical study. *Computers & Operations Research*, v. 28, pp. 767-785, 2001.

Árvores

Referências

- CHU, Y.J.; LIU, T.H. On the shortest arborescence of a directed graph, *Science Sinica*, v.14, 1965, pp.1396-1400.
- EDMONDS, J. Optimum branchings, *J. Research of the National Bureau of Standards*, 71B, 1967, pp.233-240.
- KRUSKAL Jr, J. B. On the shortest spanning subtree of a graph and the traveling salesman problem. In: *Proceedings of the American Mathematical Society*, v. 7, n. 1, pp. 48–50, 1956.
- PRIM, R. C. Shortest connection networks and some generalizations*. Bell System Technical Journal*, v. 36, pp. 1389-1401, 1957.
- RAIDL, G. R.; JULSTROM, B. A. Edge sets: an effective evolutionary coding of spanning trees. *IEEE Transactions on Evolutionary Computation*, v. 7, n. 3, pp. 225-239, 2003.
- TARJAN, R.E. Finding Optimum Branchings, *Networks*, v.7, 1977, pp.25-35.